

Contents

□ 裁剪--Clipping

□ 反走样技术-- Antialiasing

□ 消隐-- Visibility / occlusion

裁剪 -- Clipping



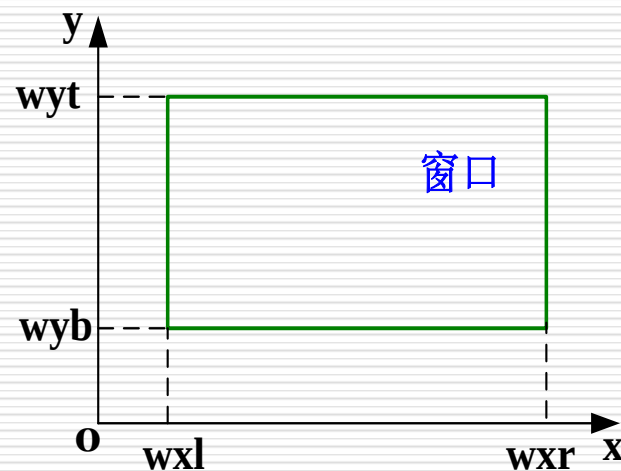
- 在二维观察中，需要在观察坐标系下对窗口进行裁剪，即只保留窗口内的那部分图形，去掉窗口外的图形。
- 假设窗口是标准矩形，即边与坐标轴平行的矩形，由上($y=wyt$)、下($y=wyb$)、左($x=wxl$)、右($x=wxr$)四条边描述。

裁剪——点的裁剪

判断点是否在窗口内：

$$wxl \leq x \leq wxr,$$

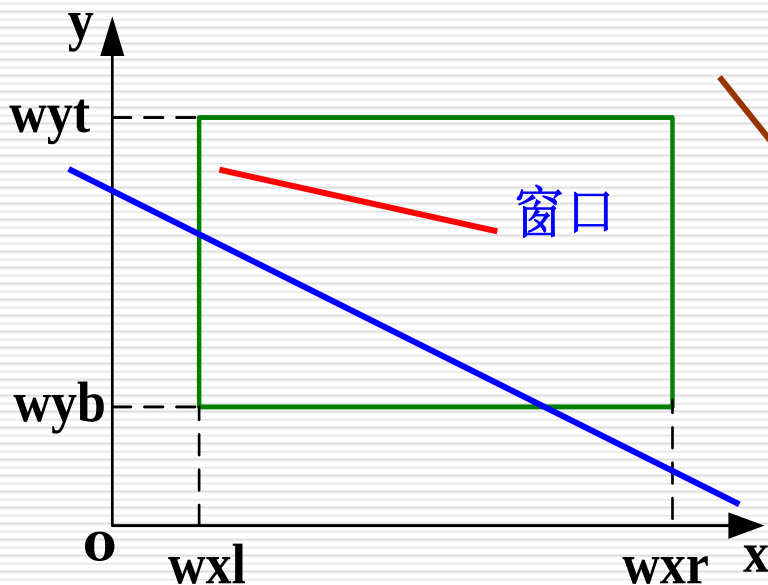
$$\text{且 } wyb \leq y \leq wyt$$



二维直线段的裁剪

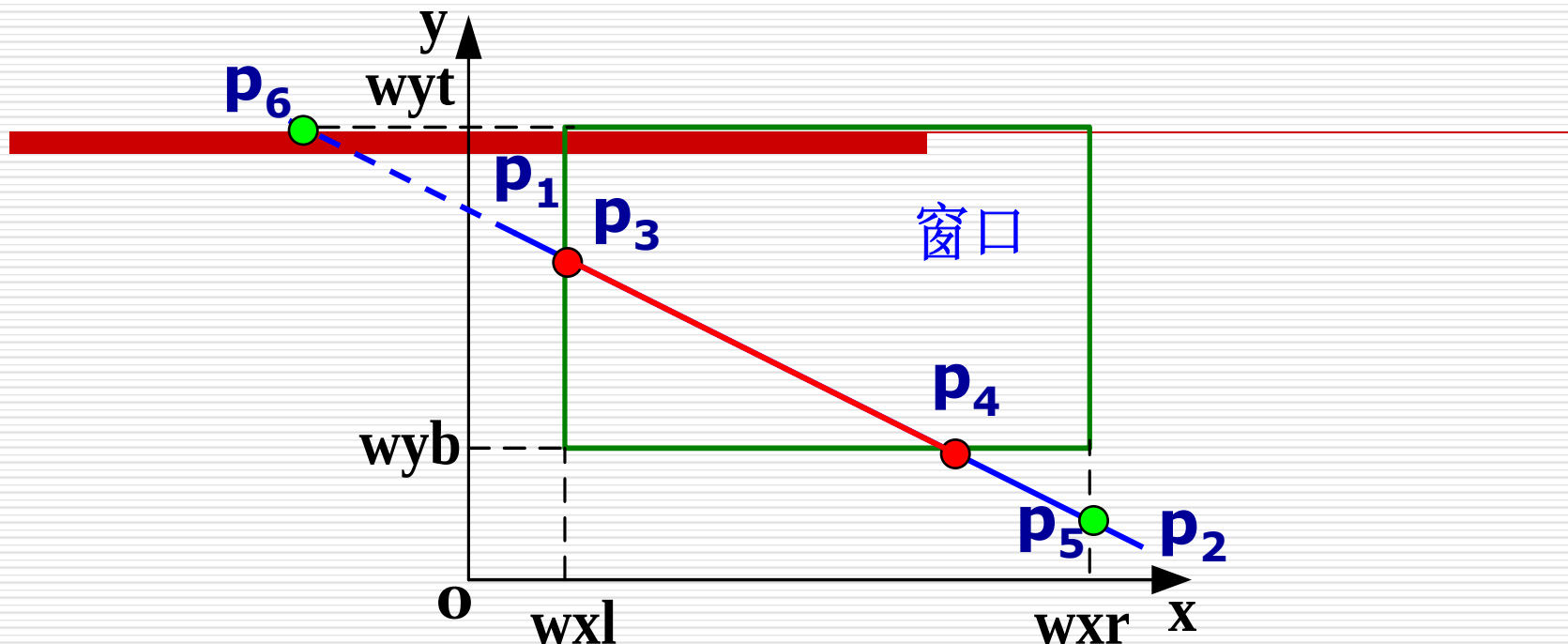
已知条件:

- (1) 窗口边界 w_{xl} , w_{xr} , w_{yb} , w_{yt} 的坐标值;
- (2) 直线段端点 p_1p_2 的坐标值 x_1, y_1, x_2, y_2 。



要裁剪一条直线段，
首先要判断它是否完全落在窗口内，或者完全落在窗口外，
若都不是，
则计算它与窗口边界的交点。

交点的分类



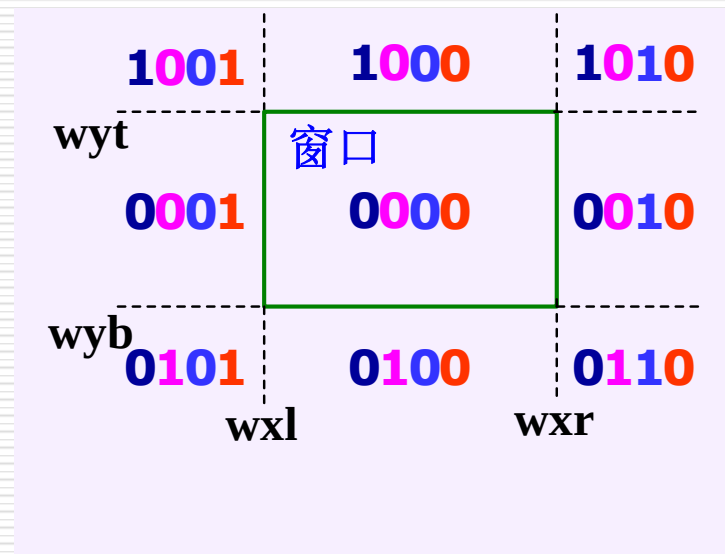
- 实交点：直线段与窗口矩形边界的交点；
- 虚交点：直线段与窗口矩形边界延长线或 直线段的延长线与窗口矩形边界及其延长线的交点。
- 根据交点计算方法不同，算法分为Cohen-Sutherland算法，中点分割算法及参数化方法等。

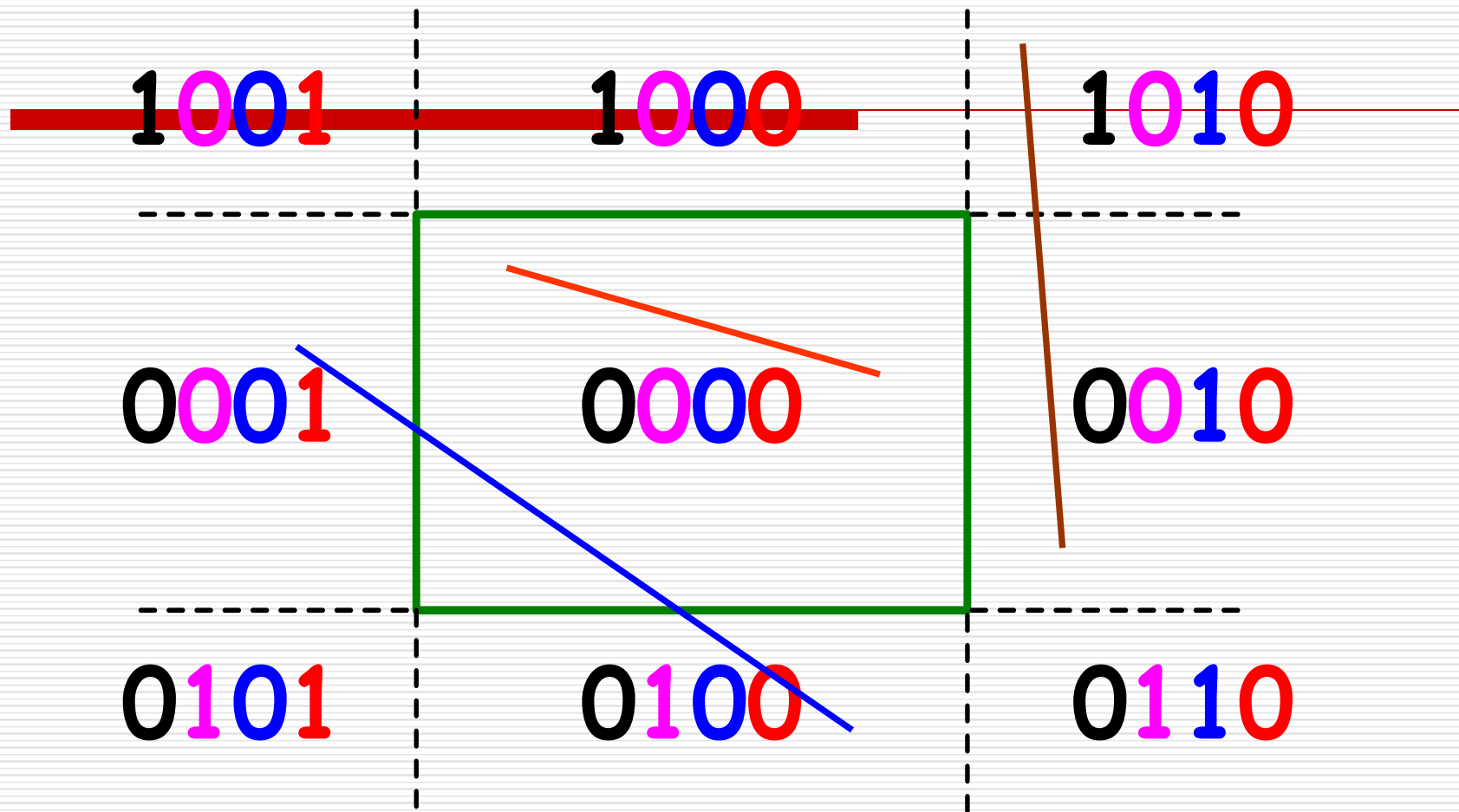
Cohen-Sutherland算法

编码：对于任一端点 (x,y) ，根据其坐标所在的区域，赋予一个4位的二进制码 $D_3D_2D_1D_0$ 。

编码规则如下：

- (1) 若 $x < wxl$ ， $D_0=1$ ，否则 $D_0=0$ ；
- (2) 若 $x > wxr$ ， $D_1=1$ ，否则 $D_1=0$ ；
- (3) 若 $y < wyb$ ， $D_2=1$ ，否则 $D_2=0$ ；
- (4) 若 $y > wyt$ ， $D_3=1$ ，否则 $D_3=0$ 。





线段的端点编码

- 区域码为：

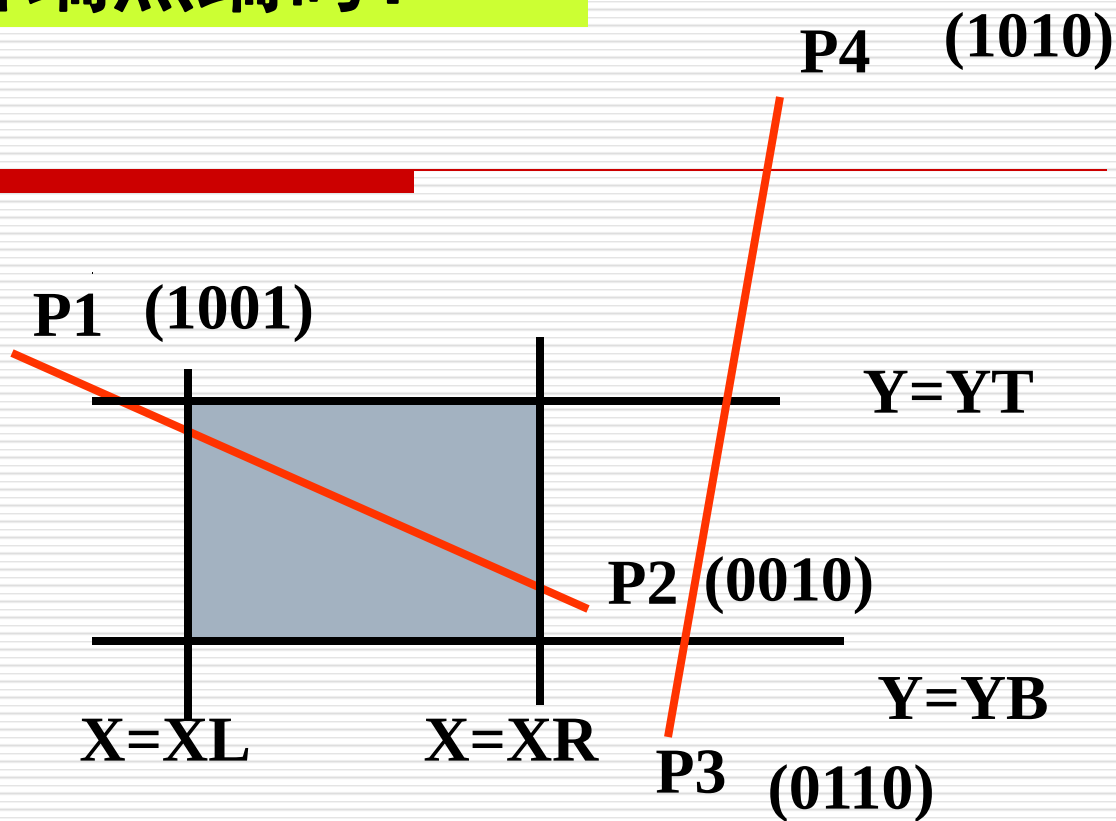
上	下	右	左
X	X	X	X

任何位赋值为1，代表端点落在相应的位置上，否则该位为0。

1001	1000	1010
0001	0000	0010
0101	0100	0110

线段的两端点编码？

上下左右
□ □ □ □



P3: 0110
P4: & 1010
0010

P1: 1001
P2: & 0010
0000

一旦给定所有的线段端点的区域码，就可以快速判断哪条直线完全在剪取窗口内，哪条直线完全在窗口外。所以得到一个**规律**：

若 P_1P_2 完全在窗口**内**： $code1=0$ ，且 $code2=0$ （即 $code1 \mid code2 == 0$ ），则“**取**”

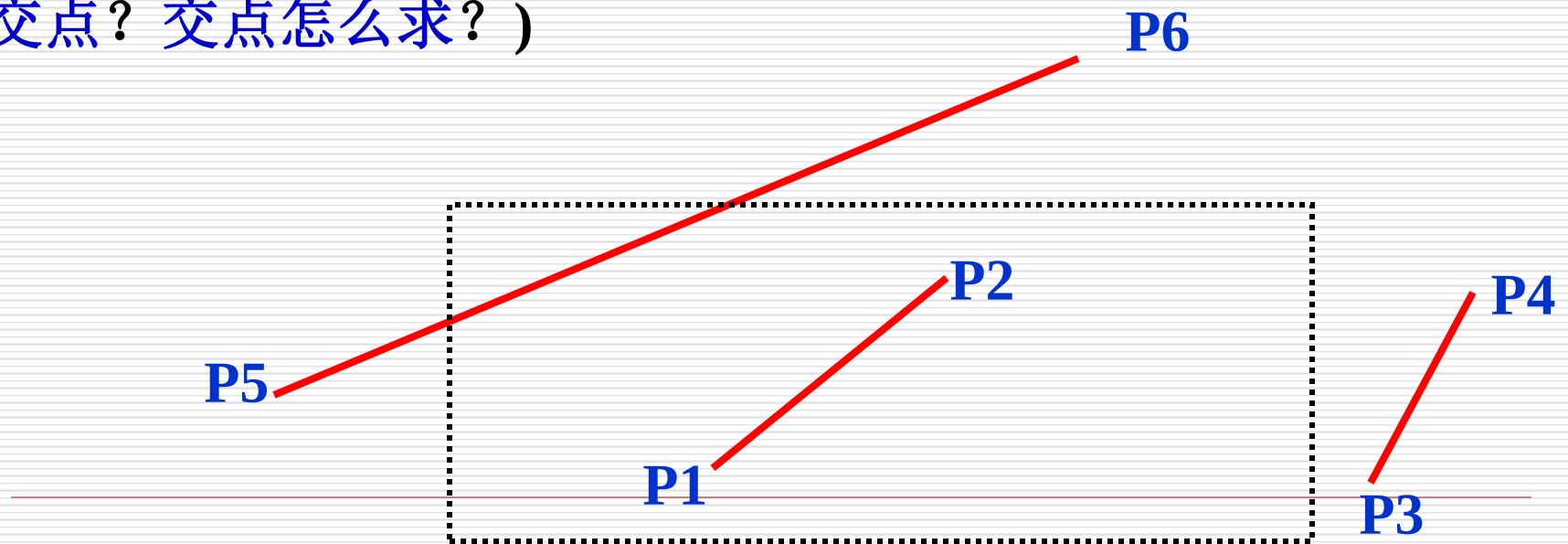
若 P_1P_2 明显在窗口**外**： $code1 \& code2 \neq 0$ ，则“**弃**”

否则，在交点处把线段分为两段。其中一段完全在窗口外，可弃之。然后对另一段重复上述处理。

（需要对直线段按交点进行分段，分段后判断直线是“简取”还是“简弃”）

Cohen-Sutherland 端点编码算法

- (1) 若线段P1P2两端点的四位编码均为0,则两端点均在窗口内,该线段完全可见,显示该线段,算法结束;
- (2) 若线段P1P2两端点的四位编码按位“与”结果为非0,则该线段完全不可见,算法结束。
- (3) 若线段两端点的四位编码按位“与”结果为0,找到P1P2在窗口外的一个端点P1(或P2),用窗口相应的边与P1P2的交点取代该端点P1(或P2),返回(1)步。(与哪条边界求交点? 交点怎么求?)



Cohen-Sutherland算法

(1) 判断

裁剪一条线段时，先求出直线段端点 p_1 和 p_2 的编码`code1`和`code2`，然后：

a. 若 $\text{code1} \mid \text{code2} = 0$ ，对直线段简取之；

b. 若 $\text{code1} \& \text{code2} \neq 0$ ，对直线段简弃之；

$D_3 D_2 D_1 D_0$
上下右左

Cohen-Sutherland算法

(2) 求交：若上述判断条件不成立，则需求出直线段与相应窗口边界的交点。

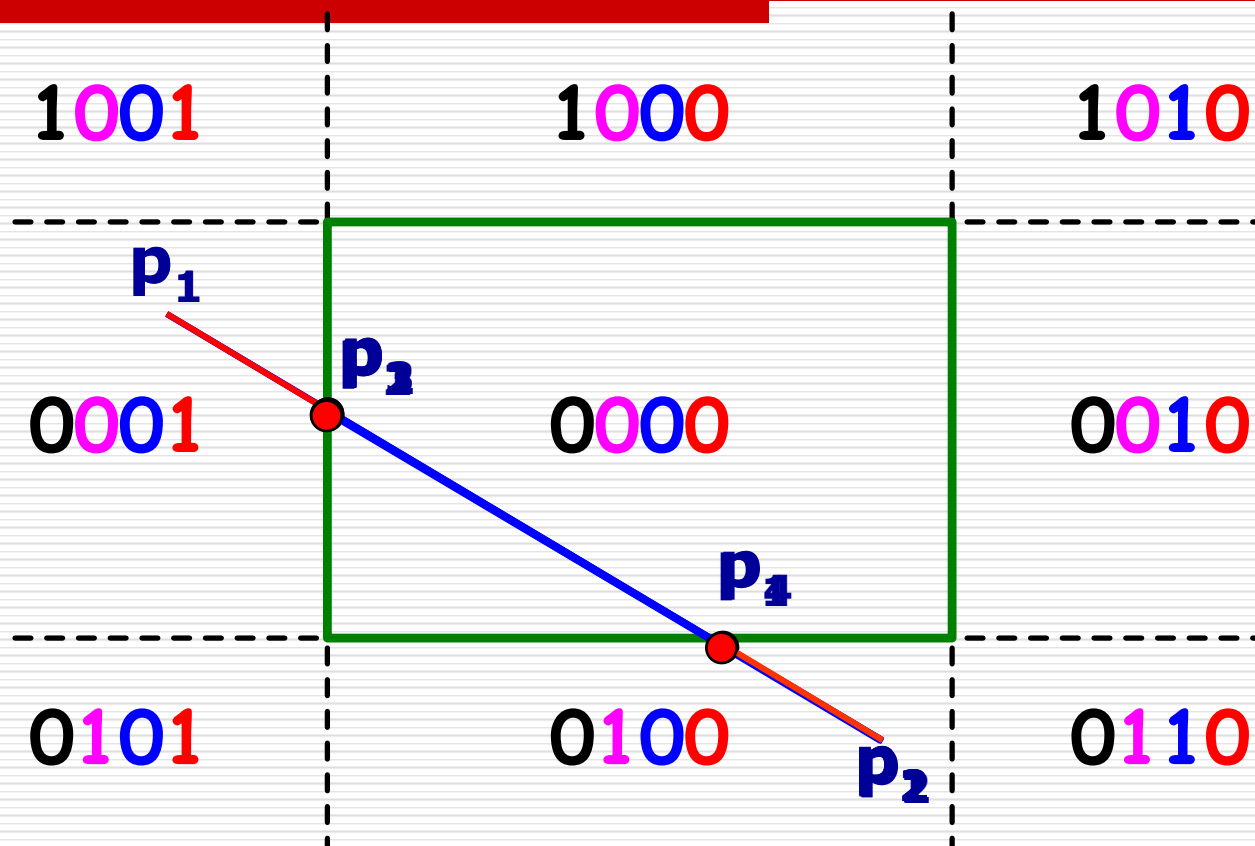
◆ 假设 **P1** 为窗口外的一个端点，从低位开始找编码为1的编码位置确定与直线段求交的窗口边界，然后求出交点。

a. 左、右边界交点的计算： $y = y_1 + k(x - x_1)$;

b. 上、下边界交点的计算： $x = x_1 + (y - y_1)/k$ 。

其中， $k = (y_2 - y_1) / (x_2 - x_1)$ 。

Cohen-Sutherland 算法

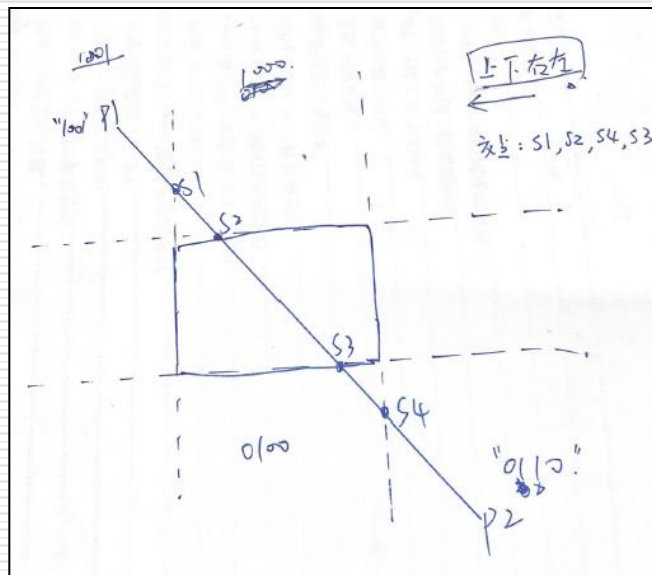


Cohen-Sutherland 端点编码算法

优点：简单，易于实现。

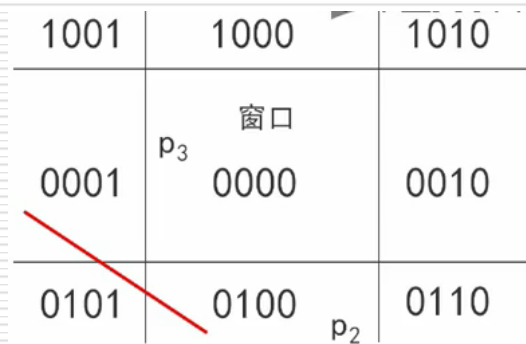
特点：用编码方法可快速判断线段的完全可见和显然不可见。

用此算法裁剪一条线段时,最多求几次交点?



Cohen-Sutherland算法

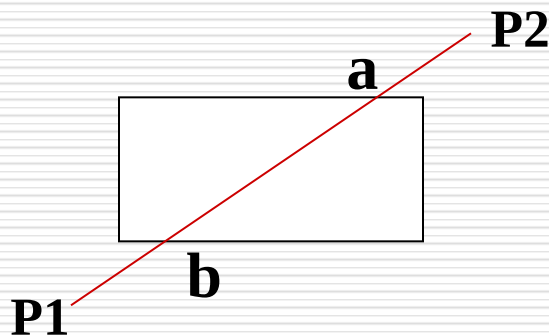
- 用编码方法实现了对完全可见和不可见直线段的快速接受和拒绝；
- 该算法在大窗口场合和窗口特别小的场合非常高效。
- 求交过程复杂，有冗余计算，并且包含浮点运算，不利于硬件实现。
 - 存在的问题：该算法计算了所有的交点之后结果确是完全舍弃！如下图中线段全部在窗口外部，但是对两段端点进行或/与运算时，需要再次取交点进行运算，被裁剪线段可能与窗口多条边计算交点，然后所得的裁剪结果却可能是全部舍弃。



中点分割（裁剪）算法（对分法）

中点分割算法是sproull和sutherland为便于用硬件实现而提出的，它与Cohen-Sutherland算法类似，首先对线段端点进行编码，并把线段与窗口的关系分为三种情况，对前两种情况，进行一样的处理；对于第三种情况，用**中点分割算法求出线段与窗口的交点**。

▪ 最远可见点

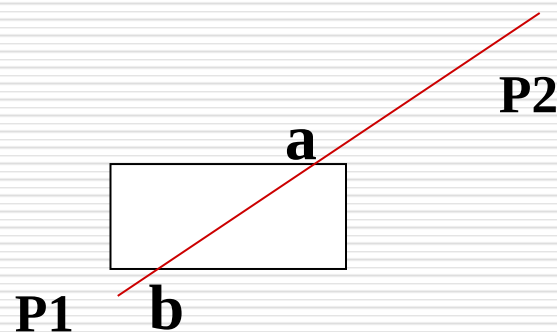


算法思想：分别寻找直线段两个端点各自的最远可见点，再显示两个可见点之间的线段。（距离端点最远且在窗口区域内的点）

对线段 P_1P_2 , 如何求 P_1 的最远可见点?

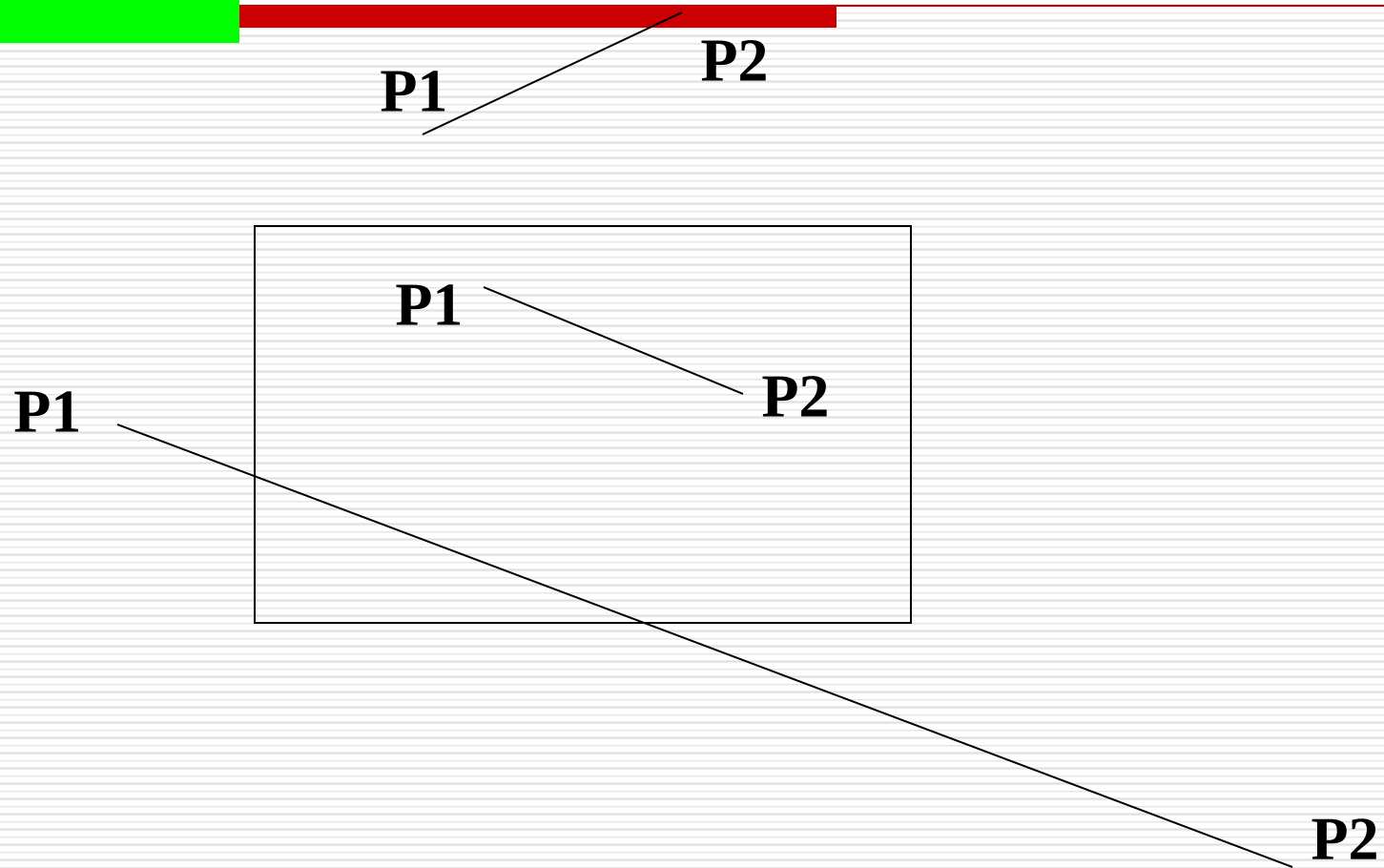
- (1) 若 P_2 在窗口内, 则 P_2 是 P_1 的最远可见点, 算法结束;
- (2) 若 P_1P_2 完全在窗口之外(完全不可见), 则算法结束;
- (3) 令 $P_a = P_1$; $P_b = P_2$;
- (4) 求 P_aP_b 的中点 P_M , 若 $P_M P_b$ 完全在窗口外, 则 $P_b \leftarrow P_M$, 返回(4)步, 直到 $|P_a P_M| < \varepsilon$ 停止, 则 P_M 即为 P_1 的最远可见点;
若 $P_M P_b$ 没有完全在窗口外, 则 $P_a \leftarrow P_M$, 返回(4)步, 直到 $|P_M P_b| < \varepsilon$ 停止, 则 P_M 即为 P_1 的最远可见点。

类似求 P_2 的最远可见点。



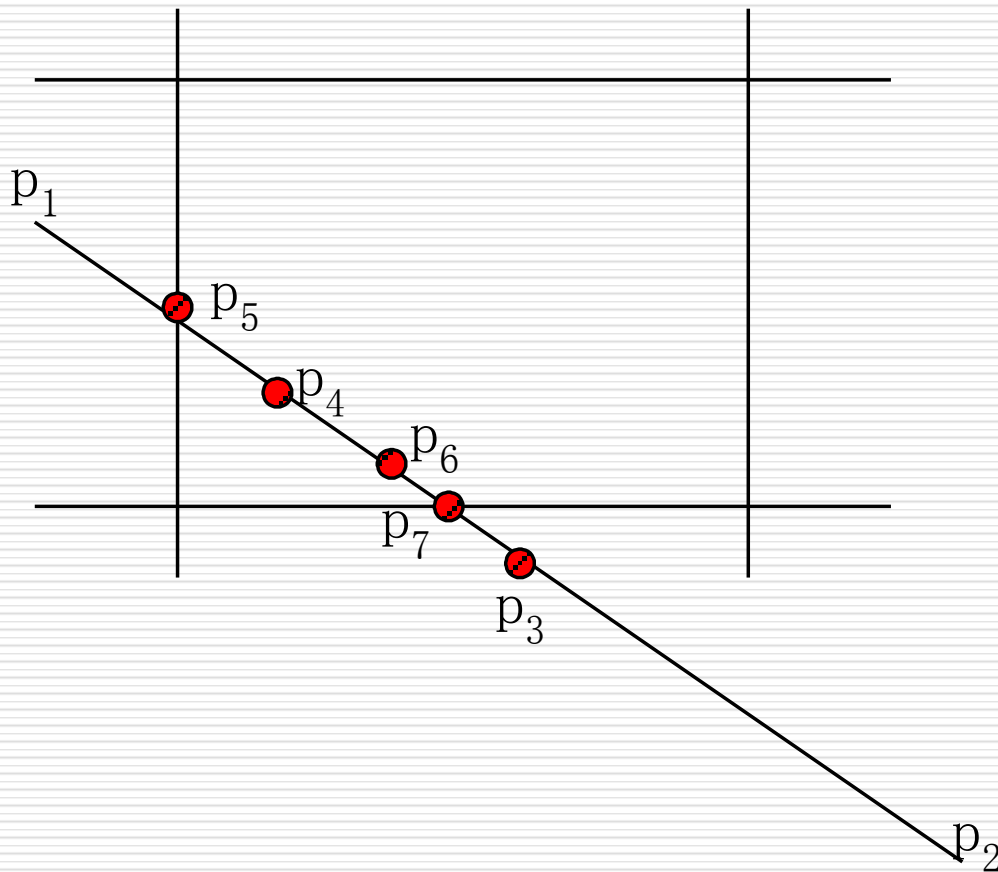
如何求 P_1 的最远可见点?

例:



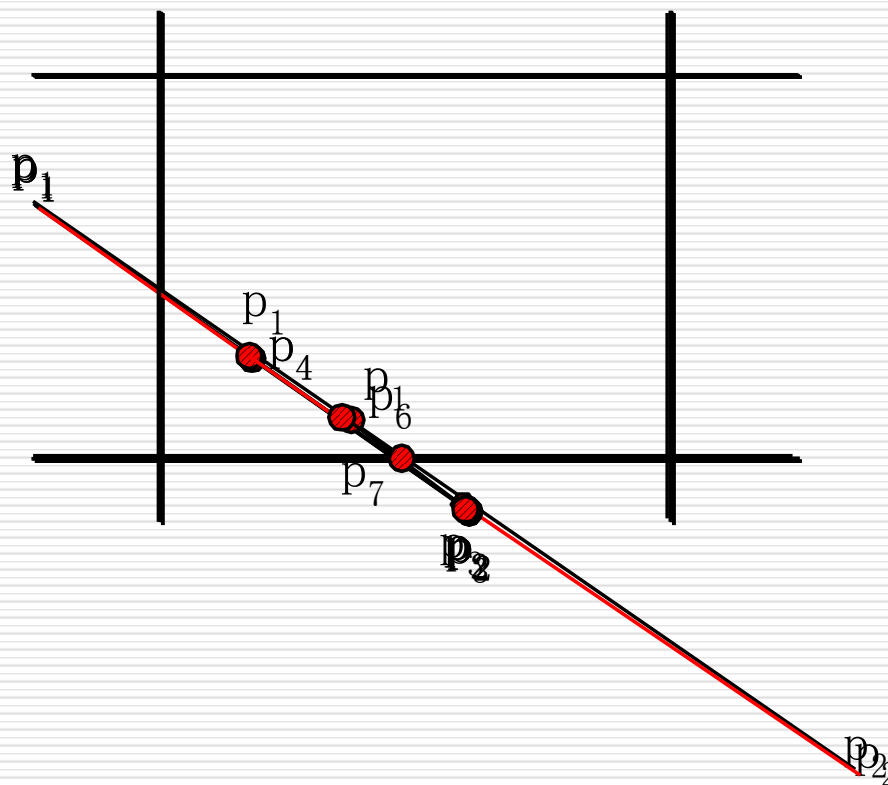
中点分割算法

□ 中点分割算法的核心思想是通过二分逼近来确定直线段与窗口的交点。



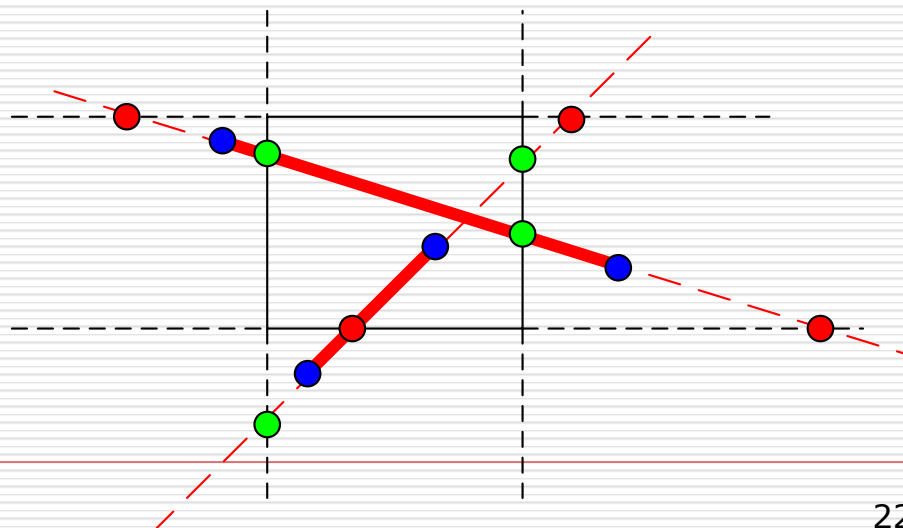
中点分割算法

- 特点：主要计算过程只用到加法或位移（除2）运算，易于硬件实现，同时适合于并行计算。



Liang-Barsky算法

- 至今仍是计算机图形学中最经典的算法之一，也是写进国内外主流《计算机图形学》教科书里的**唯一一个以中国人命名**的算法。这就叫原始创新！
- **思想**：把被裁剪的红色直线段看成是一条**有方向**的线段，把窗口的四条边分成两类：**入边和出边**。
- **重要结论**：裁剪结果的线段**起点**是直线和两条入边的交点以及始端点三个点里**最前面的一个点**；
裁剪线段的**终点**是和两条出边的交点以及端点最后面的一个点。



Liang-Barsky算法

直线的参数方程

$$\begin{aligned}x &= x_1 + u \cdot (x_2 - x_1) \\ y &= y_1 + u \cdot (y_2 - y_1)\end{aligned} \quad 0 \leq u \leq 1$$

对于直线上一点 (x, y) , 若它在窗口内则有

$$wxl \leq x_1 + u \cdot (x_2 - x_1) \leq wxr$$

$$wxb \leq y_1 + u \cdot (y_2 - y_1) \leq wyt$$

$$u \cdot (x_1 - x_2) \leq x_1 - wxl$$

$$u \cdot (x_2 - x_1) \leq wxr - x_1$$

$$u \cdot (y_1 - y_2) \leq y_1 - wyb$$

$$u \cdot (y_2 - y_1) \leq wyt - y_1$$

$$p_1 = -(x_2 - x_1) \quad q_1 = x_1 - wxl$$

$$p_2 = x_2 - x_1 \quad q_2 = wxr - x_1$$

$$p_3 = -(y_2 - y_1) \quad q_3 = y_1 - wyb$$

$$p_4 = y_2 - y_1 \quad q_4 = wyt - y_1$$

则有 $u \cdot p_k \leq q_k$

Liang-Barsky算法

由 $u \cdot p_k \leq q_k$

$$\begin{cases} \frac{q_k}{p_k} (p_k < 0) \leq u \leq \frac{q_k}{p_k} (p_k > 0) & k=1,2 \\ \frac{q_k}{p_k} (p_k < 0) \leq u \leq \frac{q_k}{p_k} (p_k > 0) & k=3,4 \\ 0 \leq u \leq 1 \end{cases}$$

线段和窗口边界一共有四个交点，根据 p_k 的符号，就知道哪两个是入交点，哪两个是出交点

当 $p_k < 0$ 时：对应入边交点

当 $p_k > 0$ 时：对应出边交点

一共四个 u 值，再加上 $u=0$ 、 $u=1$ 两个端点值，总共六个值

把 $p_k < 0$ 的两个 u 值和0比较去找最大的，把 $p_k > 0$ 的两个 u 值和1比较去找最小的，这样就得到两个端点的参数值

特殊处理1: 若 $\Delta\mathbf{x}=\mathbf{0}$, 则 $\mathbf{p}_1=\mathbf{p}_2=\mathbf{0}$

- 若 $\mathbf{q}_1<\mathbf{0}$ 或 $\mathbf{q}_2<\mathbf{0}$, 则直线段在窗口外
- 若 $\mathbf{q}_1>\mathbf{0}$ 且 $\mathbf{q}_2>\mathbf{0}$, 则按需进一步计算如下

$$\begin{array}{ll} p_1 = -(x_2 - x_1) & q_1 = x_1 - wxl \\ p_2 = x_2 - x_1 & q_2 = wxr - x_1 \\ p_3 = -(y_2 - y_1) & q_3 = y_1 - wyb \\ p_4 = y_2 - y_1 & q_4 = wyt - y_1 \end{array}$$

$$p_3 = -(y_2 - y_1) \quad q_3 = y_1 - wyb$$

$$p_4 = y_2 - y_1 \quad q_4 = wyt - y_1$$

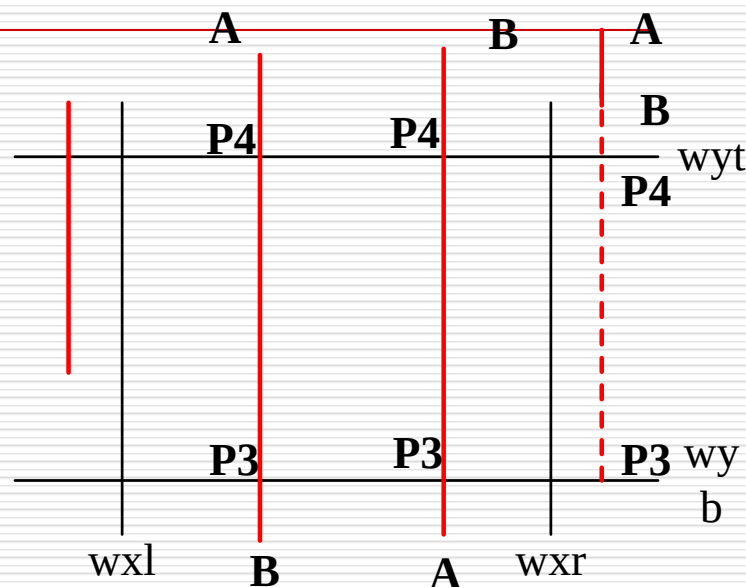
求出参数值:

$$u_3 = q_3/p_3, \quad u_4 = q_4/p_4$$

$$u_A = 0, \quad u_B = 1,$$

$$u_{\max} = \max(0, u_k \mid p_k < 0)$$

$$u_{\min} = \min(u_k \mid p_k > 0, 1)$$



(a) 直线段与窗口边界 wxl 和 wxr 平行的情况

特殊处理2: 若 $\Delta y=0$, 则 $p_3=p_4=0$

- 若 $q_3 < 0$ 或 $q_4 < 0$, 则直线段在窗口外
- 若 $q_3 > 0$ 且 $q_4 > 0$, 则按需进一步计算如下

$$\begin{array}{ll} p_1 = -(x_2 - x_1) & q_1 = x_1 - wxl \\ p_2 = x_2 - x_1 & q_2 = wxr - x_1 \\ p_3 = -(y_2 - y_1) & q_3 = y_1 - wyb \\ p_4 = y_2 - y_1 & q_4 = wyt - y_1 \end{array}$$

$$\begin{array}{ll} p_1 = -(x_2 - x_1) & q_1 = x_1 - wxl \\ p_2 = x_2 - x_1 & q_2 = wxr - x_1 \end{array}$$

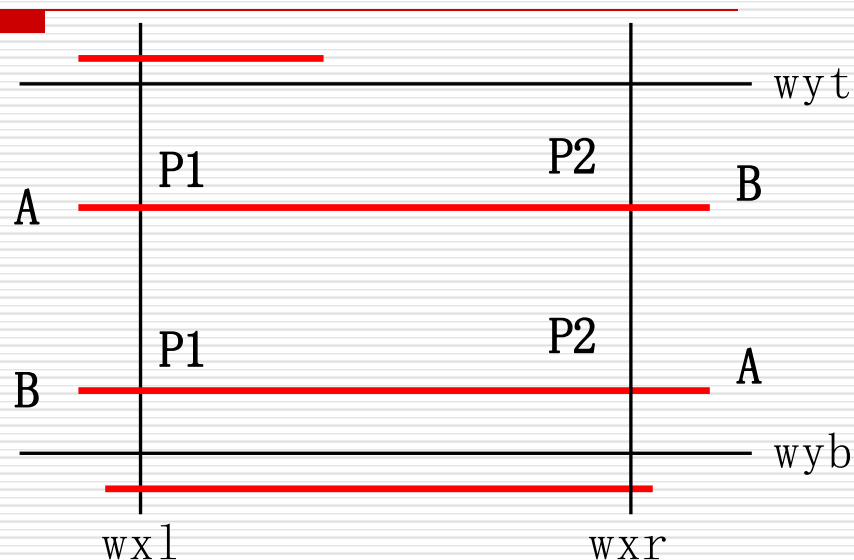
求出参数值:

$$u_1 = q_1/p_1, \quad u_2 = q_2/p_2$$

$$u_A = 0, \quad u_B = 1,$$

$$u_{\max} = \max(0, u_k \mid p_k < 0)$$

$$u_{\min} = \min(u_k \mid p_k > 0, 1)$$



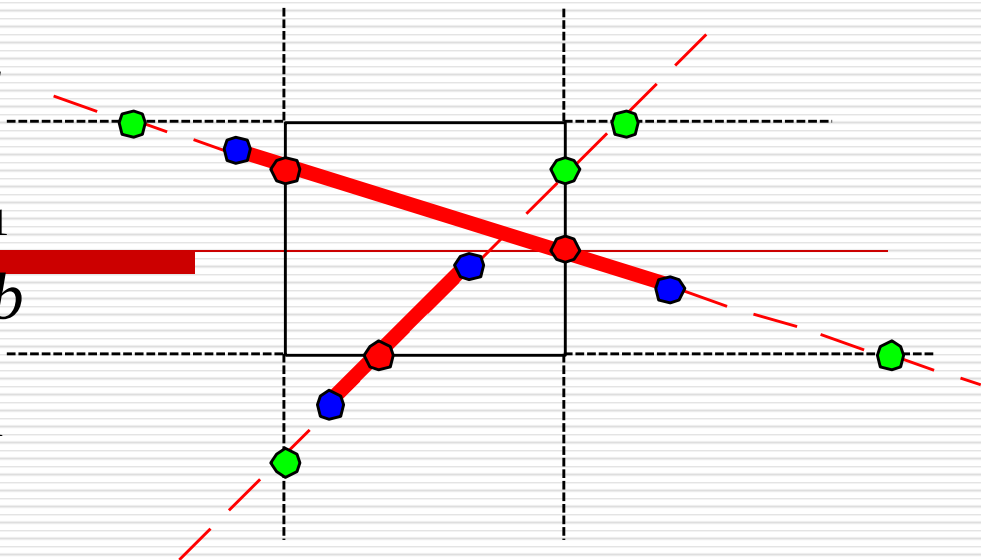
(b) 直线段与窗口边界 wyb 和 wyt 平行的情况

$$p_1 = -(x_2 - x_1) \quad q_1 = x_1 - wxl$$

$$p_2 = x_2 - x_1 \quad q_2 = wxr - x_1$$

$$p_3 = -(y_2 - y_1) \quad q_3 = y_1 - wxb$$

$$p_4 = y_2 - y_1 \quad q_4 = wyt - y_1$$



一般情况:

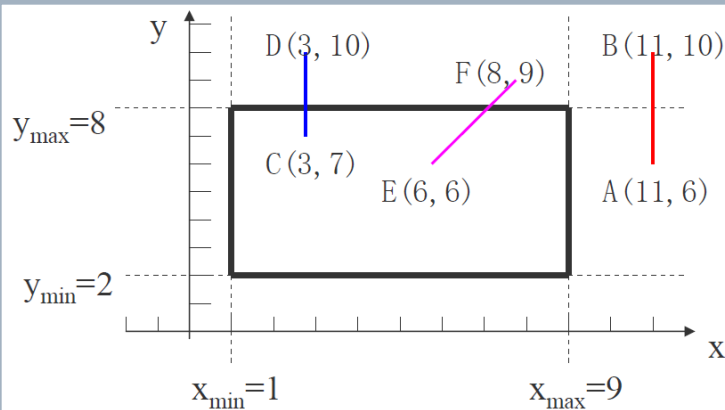
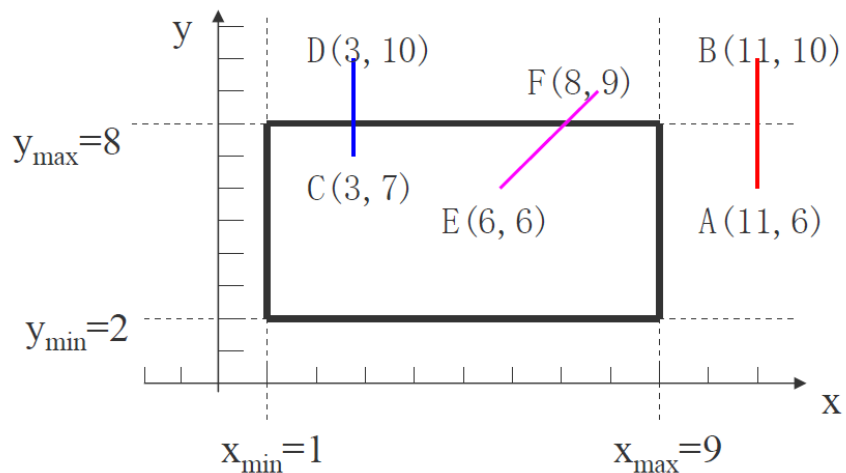
$$u_{\max} = \max(0, u_k \mid p_k < 0, u_k \mid p_k < 0)$$

$$u_{\min} = \min(u_k \mid p_k > 0, u_k \mid p_k > 0, 1)$$

算法步骤:

- (1) 输入直线段的两端点坐标: (x_1, y_1) 和 (x_2, y_2) , 以及窗口的四条边界坐标: wyt 、 wyb 、 wxl 和 wxr 。
- (2) 若 $\Delta x = 0$, 则 $p_1 = p_2 = 0$ 。此时进一步判断是否满足 $q_1 < 0$ 或 $q_2 < 0$, 若满足, 则该直线段不在窗口内, 算法转(7)。否则, 满足 $q_1 \geq 0$ 且 $q_2 \geq 0$, 则进一步计算 $u_{\max}$ 和 $u_{\min}$ 。算法转(5)。
- (3) 若 $\Delta y = 0$, 则 $p_3 = p_4 = 0$ 。此时进一步判断是否满足 $q_3 < 0$ 或 $q_4 < 0$, 若满足, 则该直线段不在窗口内, 算法转(7)。否则, 满足 $q_3 \geq 0$ 且 $q_4 \geq 0$, 则进一步计算 $u_{\max}$ 和 $u_{\min}$ 。算法转(5)。
- (4) 若上述两条均不满足, 则有 $p_k \neq 0 (k=1, 2, 3, 4)$ 。此时计算 $u_{\max}$ 和 $u_{\min}$ 。
- (5) 求得 $u_{\max}$ 和 $u_{\min}$ 后, 进行判断: 若 $u_{\max} > u_{\min}$, 则直线段在窗口外, 算法转(7)。若 $u_{\max} < u_{\min}$, 利用直线的参数方程求得直线段在窗口内的两端点坐标。
- (6) 利用直线的扫描转换算法绘制在窗口内的直线段。
- (7) 算法结束。

用Liang-Barzsky算法裁减直线段举例



令：

$$p_1 = -\Delta x$$

$$p_2 = \Delta x$$

$$p_3 = -\Delta y$$

$$p_4 = \Delta y$$

$$q_1 = x_1 - x_{\text{left}}$$

$$q_2 = x_{\text{right}} - x_1$$

$$q_3 = y_1 - y_{\text{bottom}}$$

$$q_4 = y_{\text{top}} - y_1$$

对于直线AB，有：

$$p_1 = 0 \quad q_1 = 10$$

$$p_2 = 0 \quad q_2 = -2$$

$$p_3 = -4 \quad q_3 = 4$$

$$p_4 = 4 \quad q_4 = 2$$

AB完全在右边界之右

对于直线CD，有：

$$p_1 = 0 \quad q_1 = 2$$

$$p_2 = 0 \quad q_2 = 6$$

$$p_3 = -3 \quad q_3 = 5$$

$$p_4 = 3 \quad q_4 = 1$$

$$u_3 = -5/3 \quad u_4 = 1/3$$

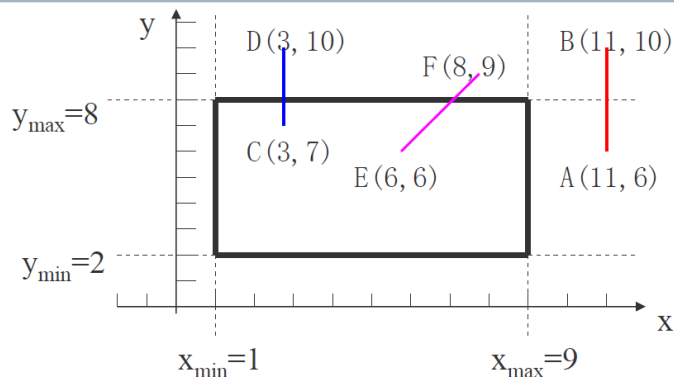
$$u_{\max} = \max(0, -5/3) = 0$$

$$u_{\min} = \min(1, 1/3) = 1/3$$

$$x = x_1 + u \cdot (x_2 - x_1)$$

$$y = y_1 + u \cdot (y_2 - y_1)$$

裁减后直线的两个端点是 (3, 7) 和 (3, 8)



对于直线EF，有：

$$p_1 = -2 \quad q_1 = 5$$

$$p_2 = 2 \quad q_2 = 3$$

$$p_3 = -3 \quad q_3 = 4$$

$$u_4 = 3/2 \quad q_4 = 2/3$$

$$u_3 = -4/3 \quad u_4 = 2/3$$

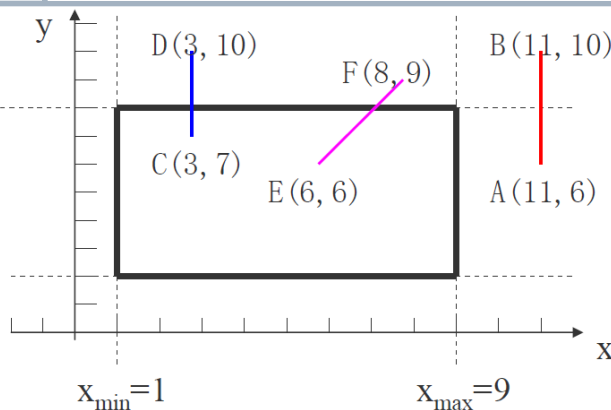
$$u_{\max} = \max(0, -5/2, -4/3) = 0$$

因此：

$$u_{\min} = \min(1, 3/2, 2/3) = 2/3$$

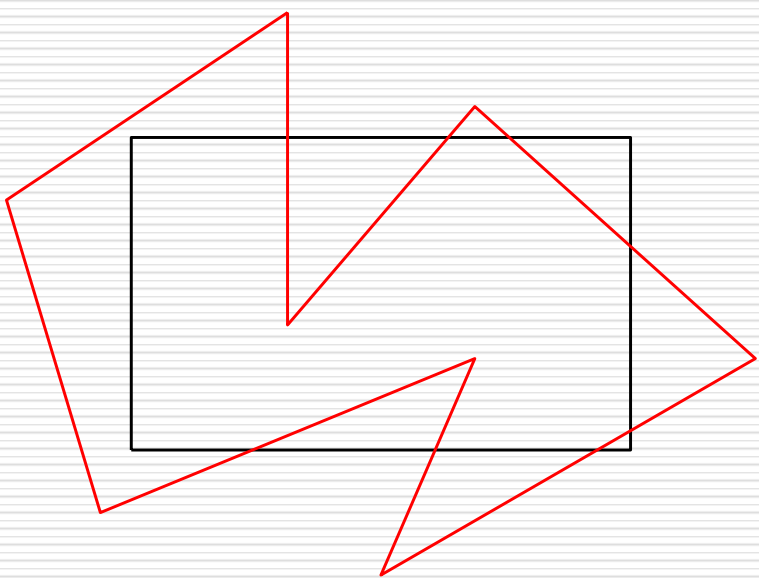
$$u_{\max} < u_{\min}$$

裁剪后的直线的两个端点是 (6, 6) 和 (7.33, 8)

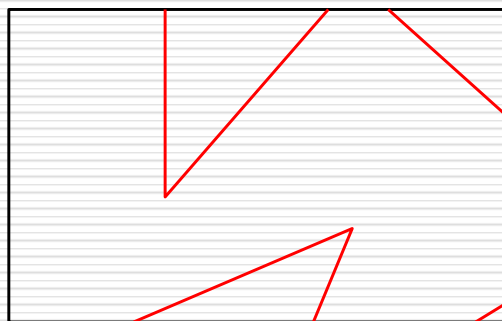


多边形的裁剪

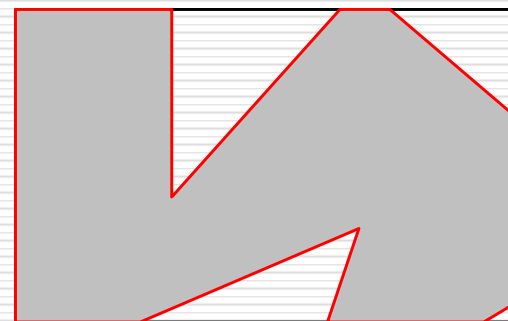
问题的提出:



(a) 裁剪前



(b) 直接采用直线段
裁剪的结果



(c) 正确的裁剪结果

- 图**(b)**得到一系列不连续的直线段!
- 实际上应该得到的是图**(c)**所示的有边界的区域。

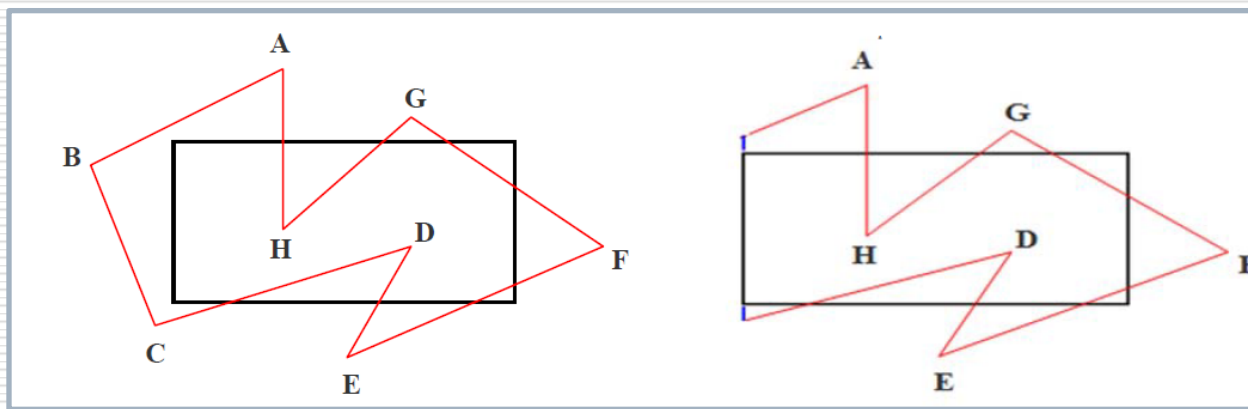
Sutherland-Hodgeman 多边形裁剪

- 基本思想：将多边形的边界作为一个整体，每次用窗口的一条边界对要裁剪的多边形进行裁剪，体现分而治之的思想。

Sutherland-Hodgeman 多边形裁剪

□ 算法实施策略1:

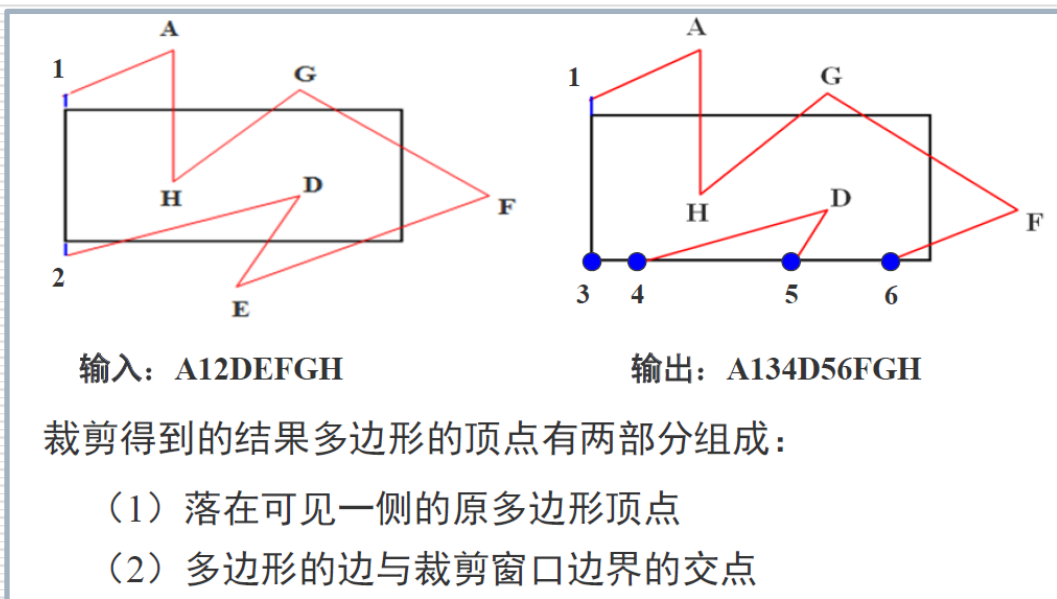
- 窗口的一条边以及延长线构成的裁剪线把平面分为两个区域，包含窗口区域的区域称为**可见侧**；不包含窗口区域的域为**不可见侧**。



Sutherland-Hodgeman 多边形裁剪

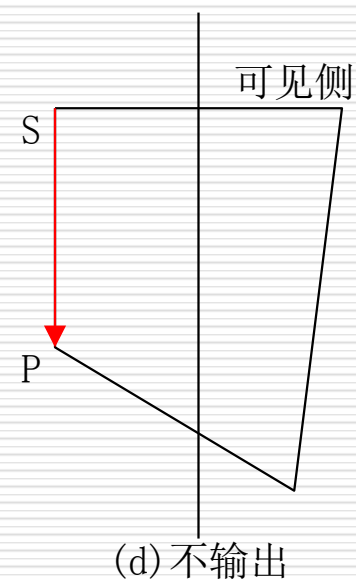
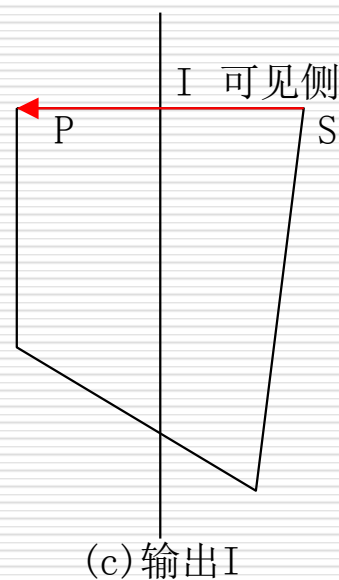
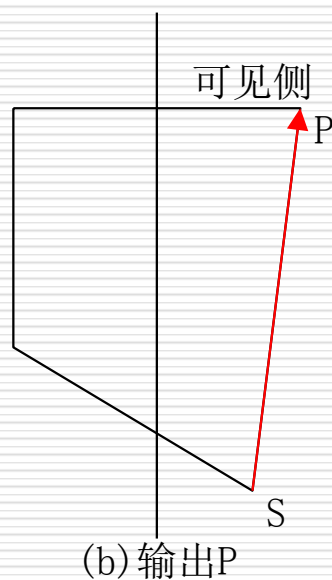
□ 算法实施策略2:

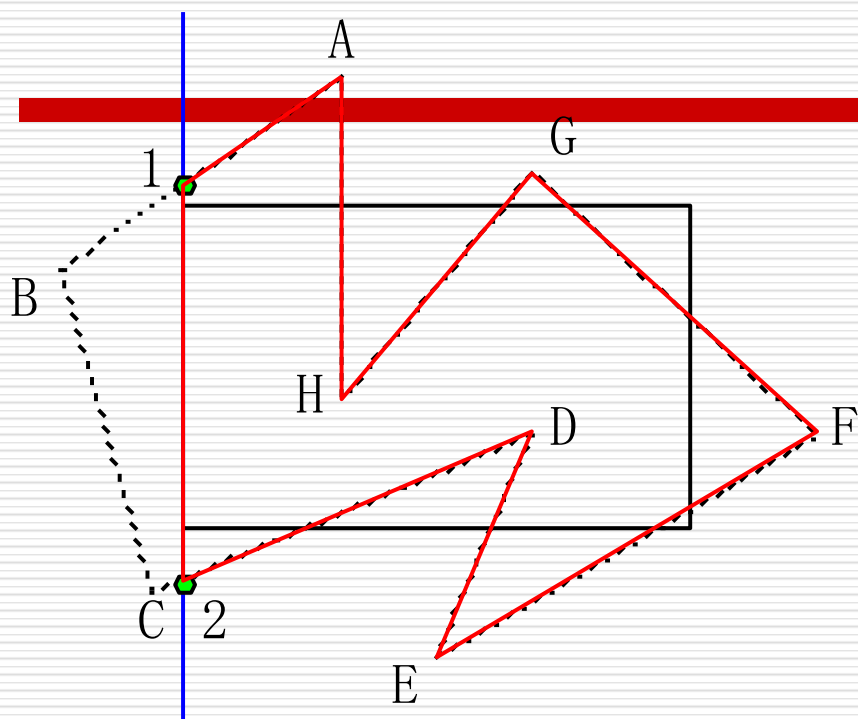
- 为窗口各边界裁剪的多边形存储**输入与输出顶点表**。在**窗口的一条裁剪边界**处理完所有顶点后，其**输出顶点表**将用窗口的下一条边界继续裁剪。



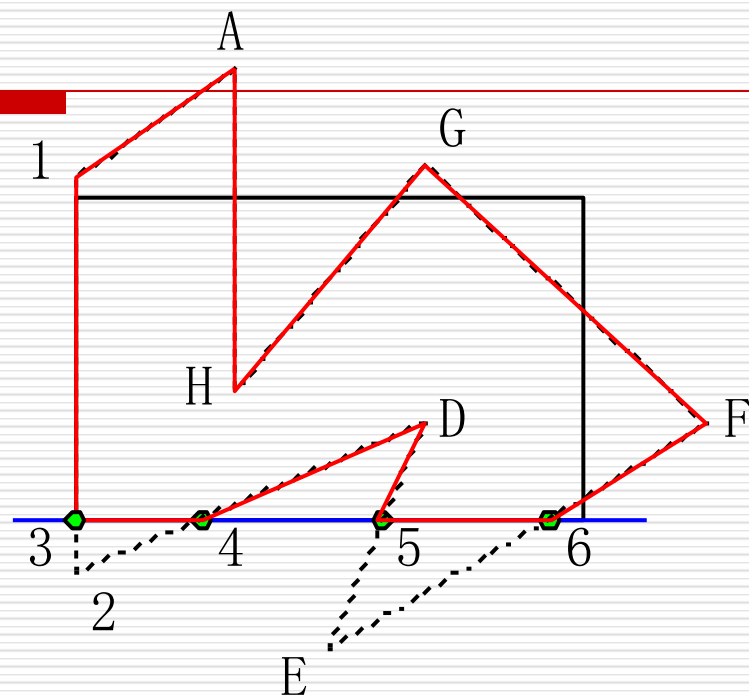
Sutherland-Hodgeman 多边形裁剪

■ 沿着多边形依次处理顶点会遇到**四种情况**：

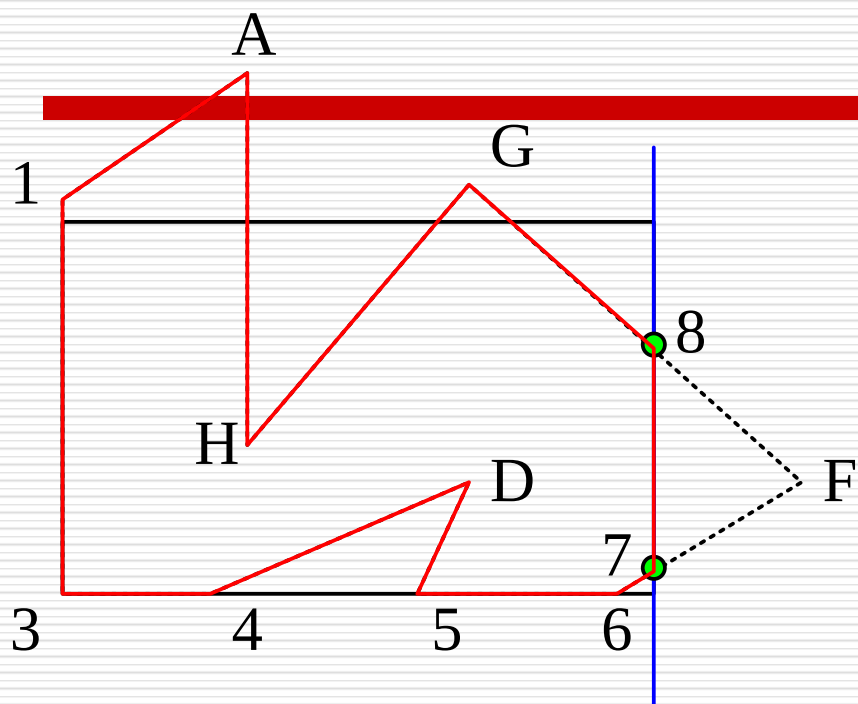




输入: ABCDEFGH
 输出: 12DEFGHA
 (a) 用左边界裁剪



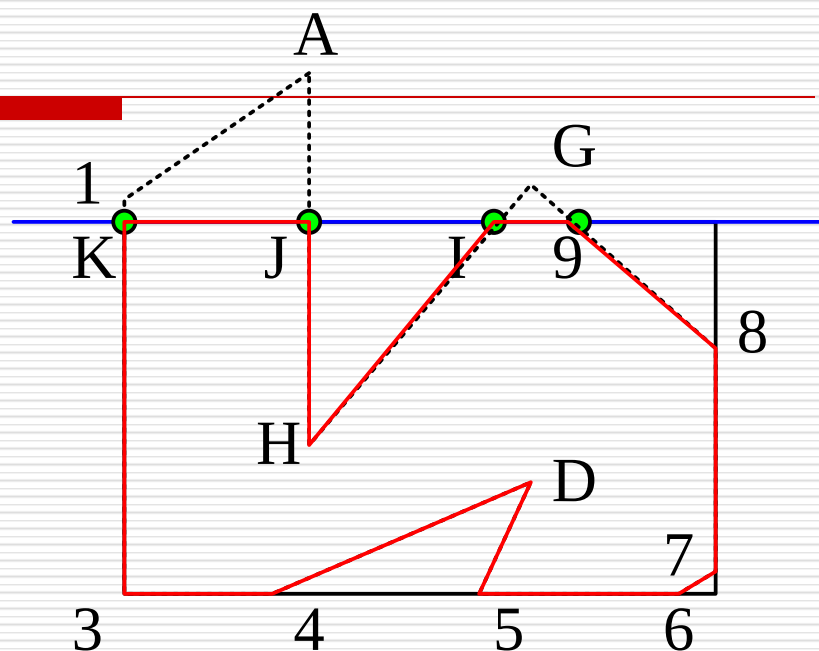
输入: 12DEFGHA
 输出: 34D56FGHA1
 (b) 用下边界裁剪



输入：34D56FGHA1

输出：4 D 5 6 7 8G H A 1 3

(c)用右边界裁剪

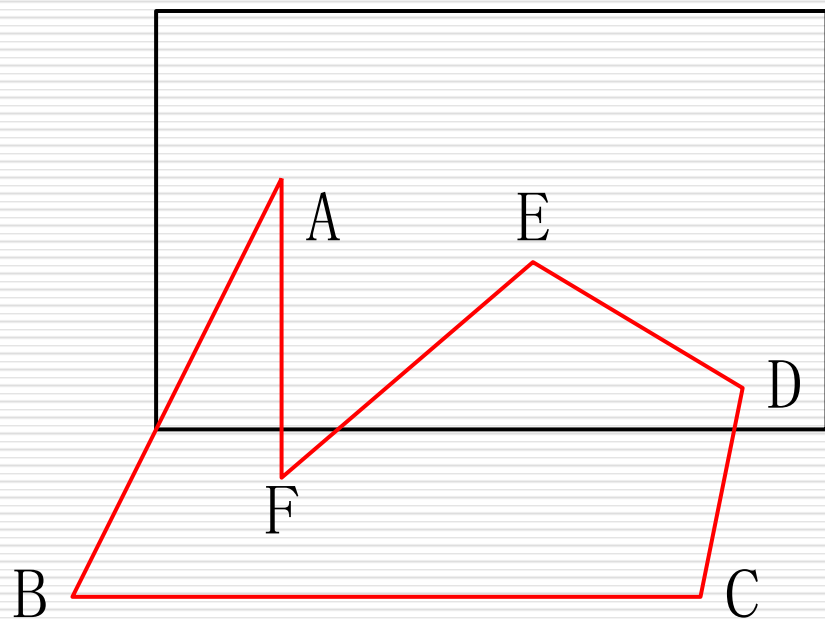


输出：D 5 6 7 8 9 IH J K3 4

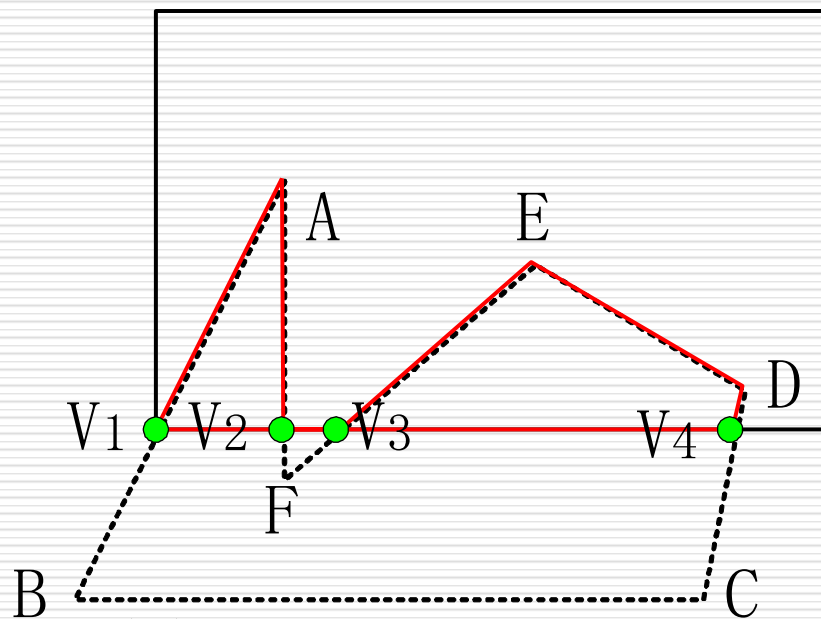
(d)用上边界裁剪

特点

凸多边形算法可以获得正确的结果，但处理凹多边形时，**只能对裁剪之后仍为一个连通图的凹多边形有效**，对下图所示凹多边形裁减之后，会产生多余的边，如下图中的 v_2v_3 。



(a) 裁剪前



(b) Sutherland-Hodgeman
算法的裁剪结果

字符裁剪

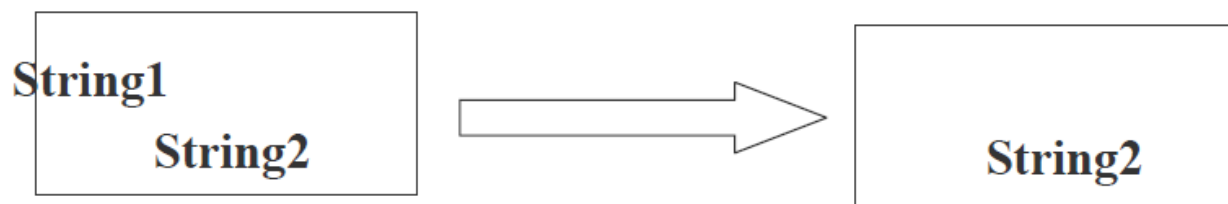
字符裁剪

文字裁剪的策略包括几种：

- 串精度裁剪
- 字符精度裁剪
- 笔划、象素精度裁剪

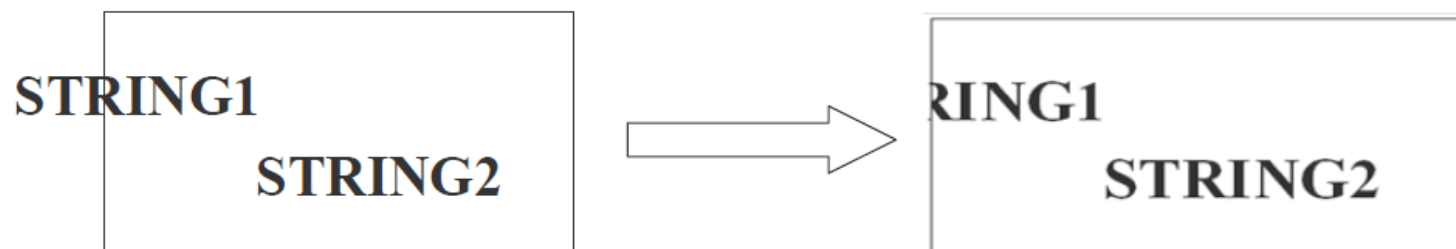
1、串精度裁剪

当字符串中的所有字符都在裁剪窗口内时，就全部保留它，否则舍弃整个字符串



3、笔画/像素精度裁剪

将笔划分解成直线段对窗口作裁剪。需判断字符串中各字符的哪些像素、笔划的哪一部分在窗口内，保留窗口内部分，裁剪掉窗口外的部分

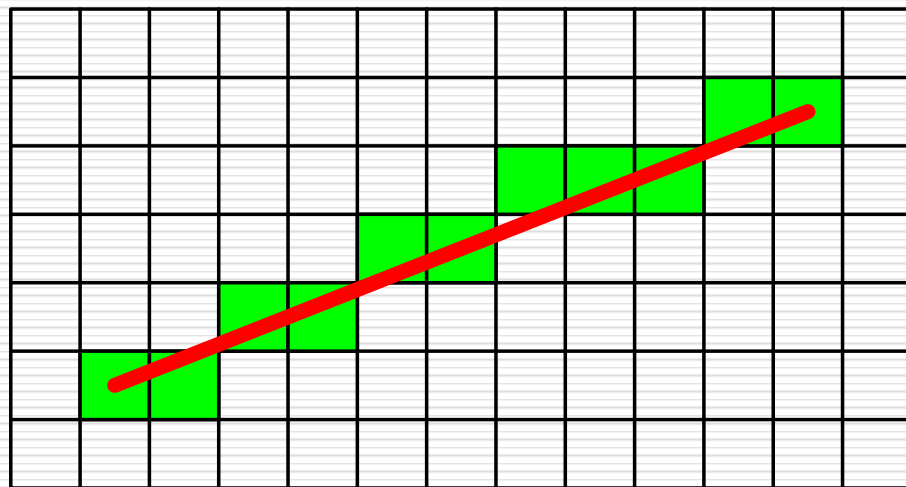


反走样

Antialiasing

走样--Aliasing

□ 用离散量表示连续量引起的失真，就叫做走样（Aliasing）。

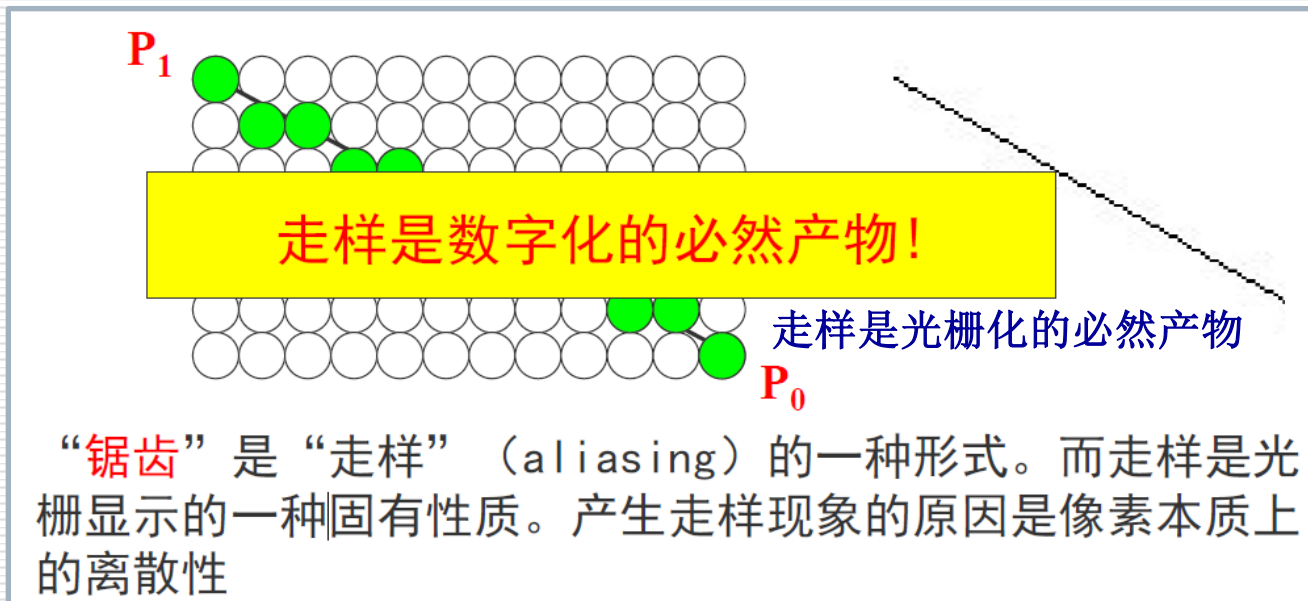


绘制图形时，具有明显的“锯齿形”（阶梯状），即会发生走样现象

图： 绘制直线时的走样现象

走样的原因

数学意义上的图形是由无限多个连续的、面积为零的点构成；但在光栅显示器上，用有限多个离散的，具有一定面积的像素来近似地表示他们。

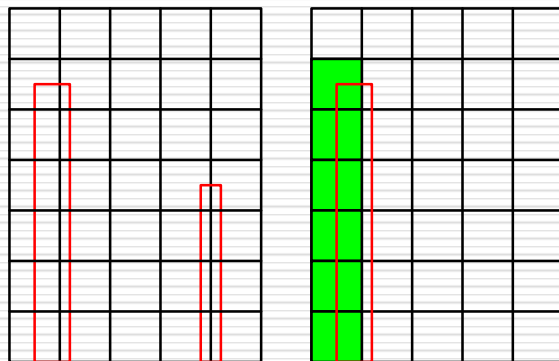


走样现象

走样现象：

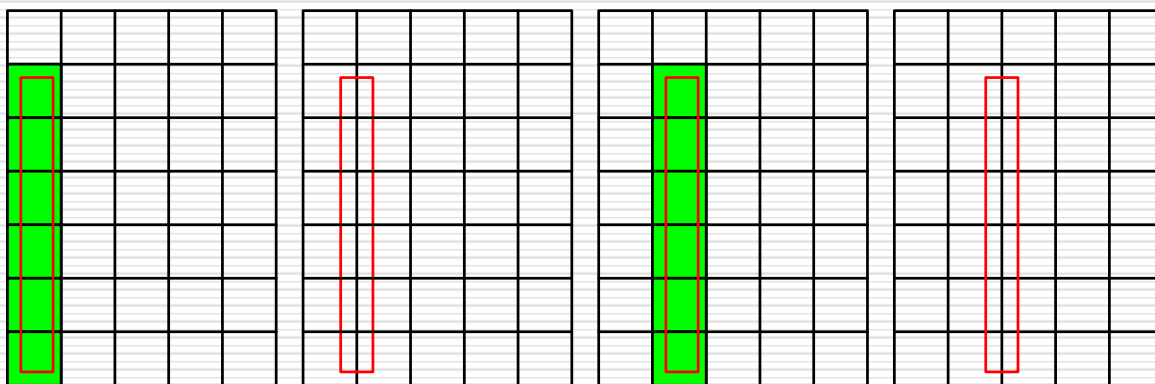
- 一是光栅图形产生的**阶梯形**的边界。
- 图形**细节失真**（图形中那些比像素更窄的细节变宽）。
- 二是**小物体由于“走样”而消失**。图形中包含相对微小的物体时，这些物体在静态图形中容易被丢弃或忽略。另外在动画序列中**时隐时现**，产生**闪烁**。

例子 丢失细节与运动图形的闪烁，如下 细小的矩形



静态图像：
图形中比像素更窄的细节
丢失（如右边细小矩形）。
左边图形覆盖了某些像素
中心，被显示出来。

(a)需显示的矩形 (b)光栅化的显示结果



(a)显示

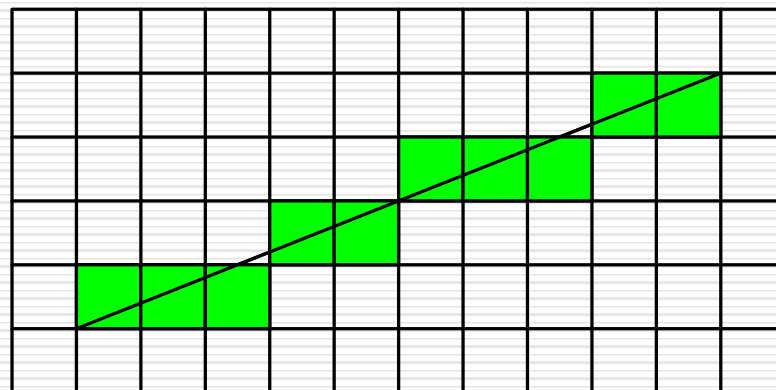
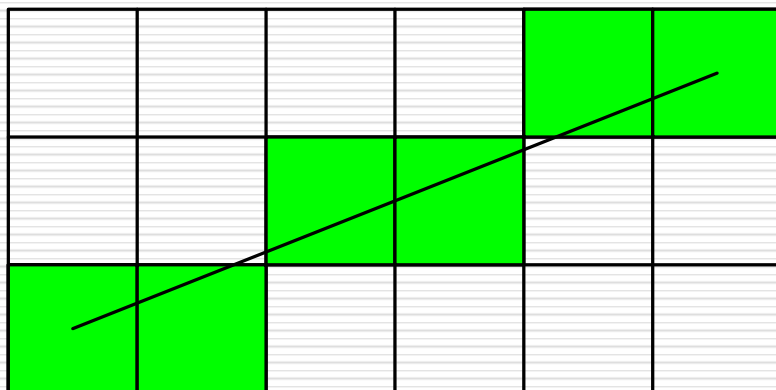
(b)不显示

(c)显示

(d)不显示

动画序列：矩形从左向右移动，当其覆盖某些像素中心时，矩形被显示出来，当没有覆盖像素中心时，矩形不被显示

- 用于减少或消除这种效果的技术，称为反走样（**antialiasing**，图形保真）。
- 一种简单方法：提高分辨率
- ✓ 采用分辨率更高的显示设备，对解决走样现象有所帮助，因为可以使锯齿相对物体会更小一些。分辨率在x方向和y方向分别提高一倍，阶梯程度减小一倍，但是以**4倍**的存储器代价和**2倍**的扫描转换时间获得的，受到硬件条件的限制。



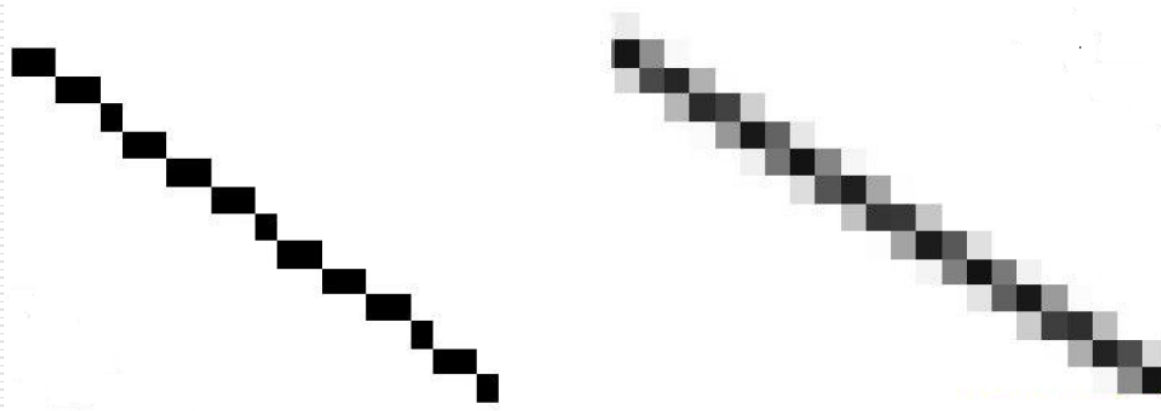
分辨率提高到原来的**2×2倍**

反走样- Antialiasing

反走样技术涉及到某种形式的“模糊”来产生更平滑的图像

对于在白色背景中的黑色矩形，通过在矩形的边界附近掺入一些灰色像素，可以柔化从黑到白的尖锐变化

从远处观察这幅图像时，人眼能够把这些缓和变化的暗影融合在一起，从而看到了更加平滑的边界



反走样方法

- 提高分辨率
- 过取样 (**supersampling**)，或超采样、后滤波
- 区域采样 (**area sampling**)，或前滤波

反走样——过取样 (super sampling)

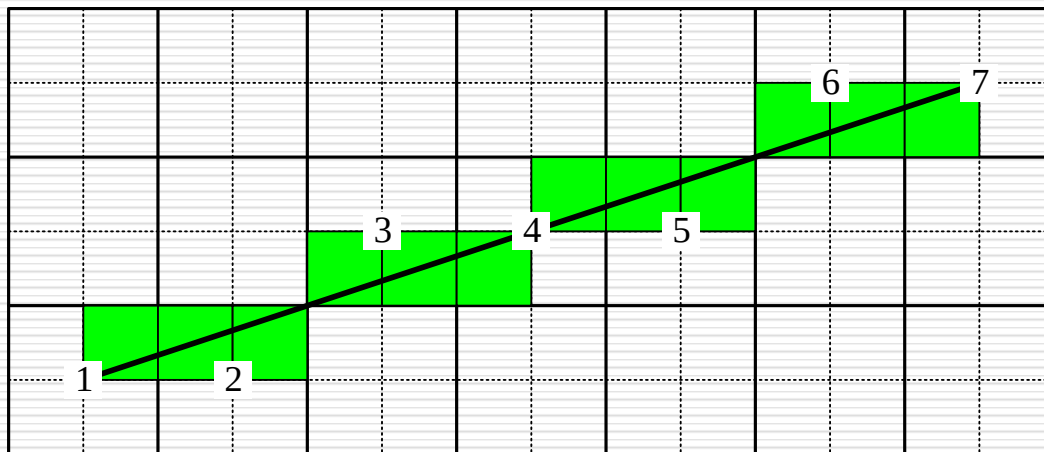
□ 过取样：在高于显示分辨率的较高分辨率下用点取样方法计算，然后对几个像素的属性进行平均得到较低分辨率下的像素属性。

对几个像素属性进行平均

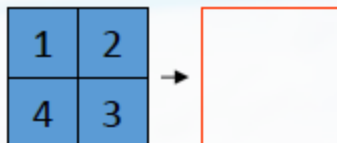
高分辨率下取样计算 ——→ 得到低分辨率下的像素属性

□ 简单过取样

在x, y方向把分辨率都提高一倍, 使每个像素对应4个子像素, 然后扫描转换求得各子像素的颜色亮度, 再对4个像素的颜色亮度进行平均, 得到较低分辨率下的像素颜色亮度, 从而减弱锯齿感。



显示分辨率 $m \times n$,
显示窗口分为 $2m \times 2n$ 个
子像素



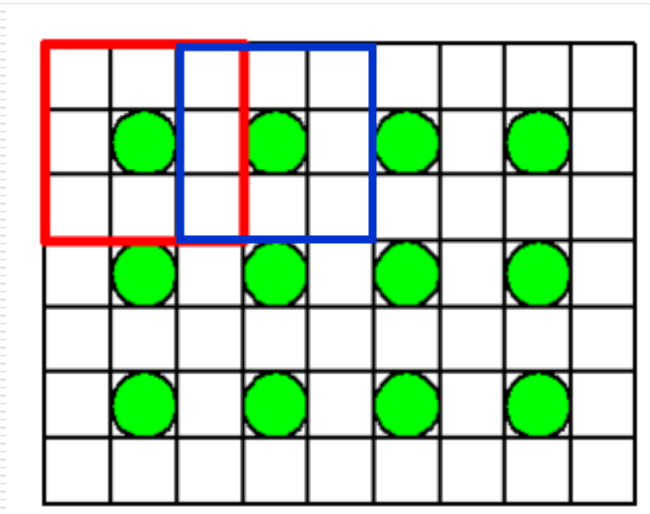
1~4像素亮度等级的和

4

(亮度等级:0~4共五级)

- **Weakness:** 相邻像素之间不互相影响, 容易产生色泽不均匀问题。

□ **重叠过取样：** 在对一个像素点进行着色处理时，同时对其周围的多个像素的子像素进行采样，而且相邻点的像素模板之间是有重复的。具体的，显示分辨率为 $m*n$ ，首先将显示窗口划分为 $(2m+1)*(2n+1)$ 个子像素，然后扫描转换求得各子像素的颜色值，对位于像素中心及四周的9个子像素的颜色值进行平均。相邻点的像素模板之间是有重复的，使得相邻像素的颜色亮度差减小。



显示分辨率 $m*n$ ，

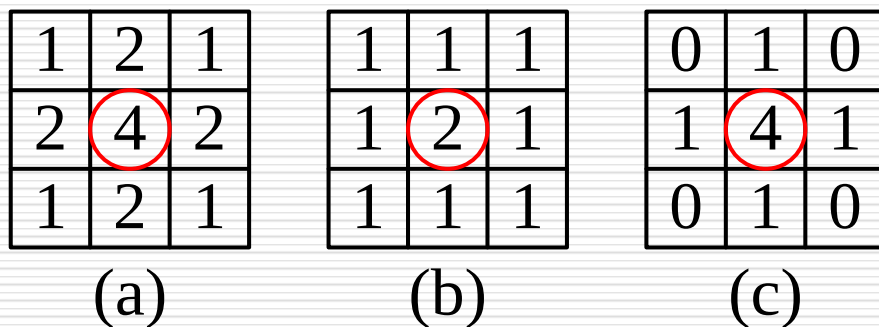
显示窗口分为 $(2m+1)*(2n+1)$ 个子像素

1~9像素亮度等级的和

9

□ 基于加权模板的过取样

前面在确定像素的亮度时，仅仅是对所有子像素的亮度进行简单的平均。更常见的做法是给接近像素中心的子像素赋予较大的权值，即对所有子像素的亮度进行加权平均。

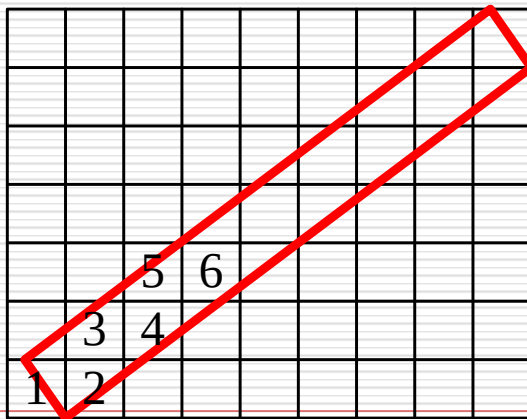


图：常用的加权模板

(这3个模板都是中心像素权值最高，4个角上像素权值更低。)

反走样——简单的区域采样

- 在扫描转化算法中都是假定像素是数学上的一个点，但实际上像素是一个有限区域，所以屏幕上所画的直线不是数学意义上没有宽度的理想线段，而是宽度为至少一个像素单位的线条。如下图，可将直线段看做具有一定宽度的狭长矩形。因此绘制这条直线时，所有与其相交的像素都可采用适当的亮度来进行显示。
- 根据图形在每个像素点上的覆盖程度来确定像素点的最终亮度，此时将像素点当成了一个有面积的平面区域而并非一个点，这种方法称之为区域采样。当直线段与像素有交时，求出两者相交区域的面积，然后根据相交区域面积大小确定该像素的亮度值。



图： 有宽度的直线段，区域取样产生模糊边界，减轻锯齿效应

反走样——简单的区域采样

如何计算直线段与像素相交区域的面积？

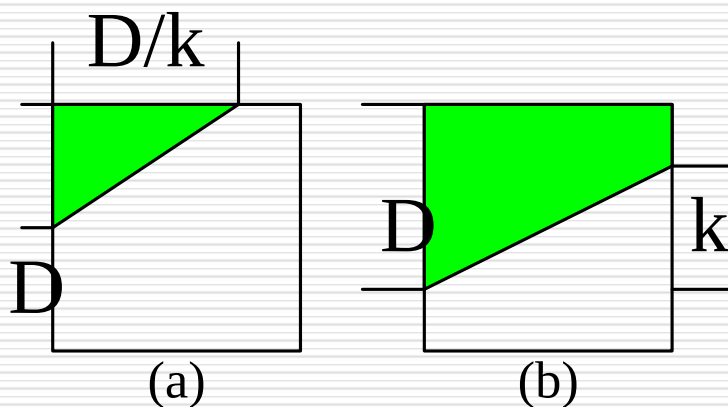


图5-54 重叠区域面积的计算

反走样——简单的区域采样

- 可以利用一种求相交区域的近似面积的离散计算方法：
 - (1)将屏幕像素分割成 n 个更小的子像素，
 - (2)计算中心落在直线段内的子像素的个数 m ，
 - (3) m/n 为线段与像素相交区域面积的近似值。
- 直线段对一个像素亮度的贡献与两者重叠区域的面积成正比。
- 相同面积的重叠区域对像素的贡献相同。

简单的区域采样的两个缺点

- 1、像素的亮度与相交区域的面积成正比，而与相交区域落在像素内的位置无关，这仍然会导致锯齿效应
- 2、直线条上沿理想直线方向的相邻两个像素有时会有较大的灰度差

- 克服上述两个缺陷的方法是采用加权区域采样（自行了解）

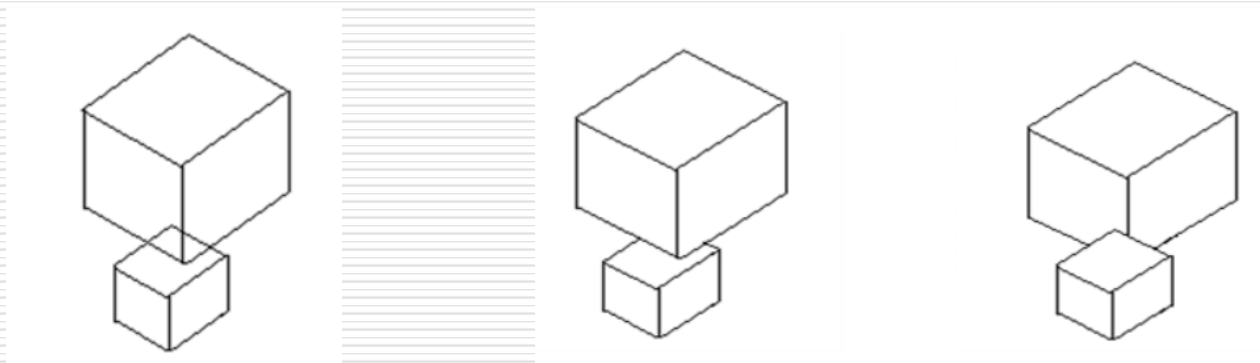
消隱

消隐

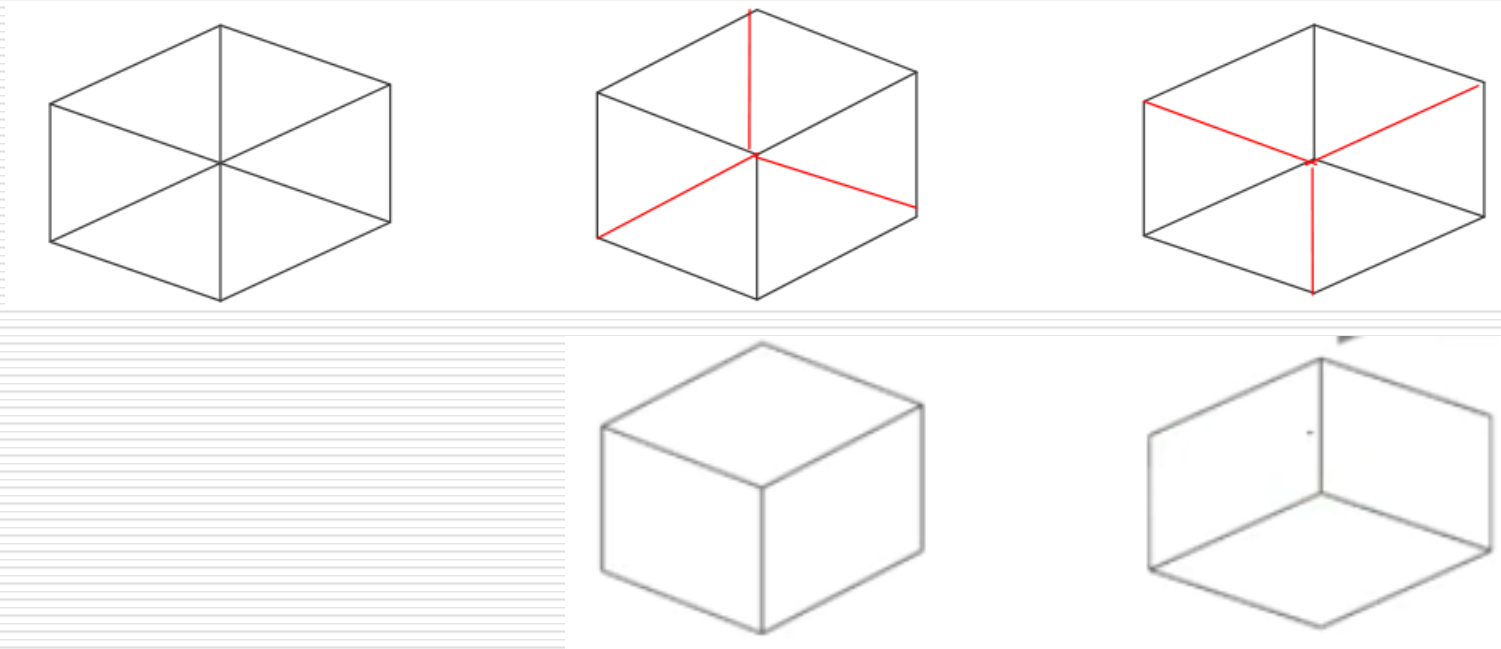
- 基本概念及分类
- 画家算法
- 深度缓冲器算法

基本概念

- 当我们观察空间任何一个不透明的物体时，只能看到该物体朝向我们的那些表面，其余的表面由于被物体所遮挡我们看不到。
- 如果把可见和不可见的线都画出来，对视觉会造成多义性。



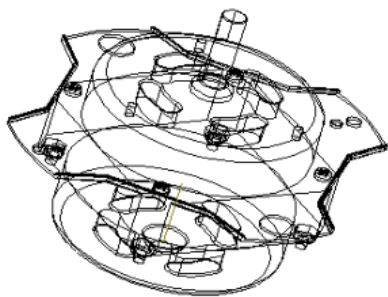
基本概念



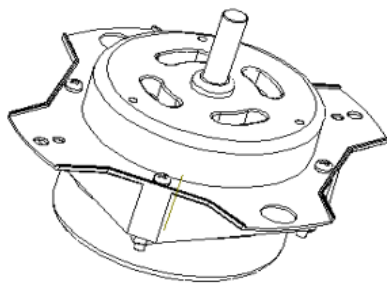
- 要消除二义性，就必须在绘制三维图形时消除被遮挡的不可见的线或面，习惯上称作消除隐藏线和隐藏面，简称为**消隐**。

基本概念

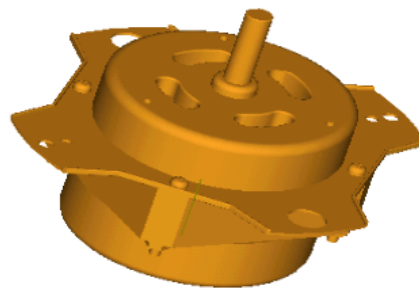
- 要绘制出意义明确的、富有真实感的立体图形，首先必须消去形体中的不可见部分，而只在图形中表现可见部分。
- 如下机械零件图，若把其中不可见的部分都画出来了，视觉上就显得比较乱。经过消隐得到的图更接近真实的机械零件。



线框图



消隐图

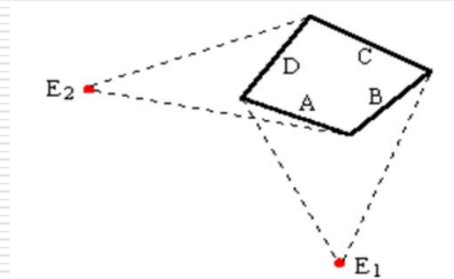


真实感图形

基本概念

- 到目前为止，虽然已有数十种算法被提出来了，但是由于物体的形状、大小、相对位置等因素千变万化，因此至今它仍吸引人们作出不懈的努力去探索更好的算法。
- 消隐不仅与消隐对象有关，还与观察者的位置有关。

物体的消隐或隐藏线面的消除：在给定视点和视线方向后，决定场景中哪些物体的表面是可见的，哪些是被遮挡不可见的。



消隐分类

1、按消隐对象分类

(1) 线消隐

消隐对象是物体上的边，消除的是物体上不可见的边

(2) 面消隐

消隐对象是物体上的面，消除的是物体上不可见的面，通常做真实感图形消隐时用面消隐

消隐分类

2、按消隐空间分类

(1) 物体空间的消隐算法

以场景中的物体为处理单元。假设场景中有 k 个物体，将其中一个物体与其余 $k-1$ 个物体逐一比较，仅显示它可见表面以达到消隐的目的

此类算法通常用于线框图的消隐！

```
for (场景中的每一个物体)
{ 将该物体与场景中的其它物体进行比较，确定其表面的可见部分；
  显示该物体表面的可见部分；
}
```

□ 如光线投射算法、**Roberts**算法等。

消隐分类

(2) 图像空间的消隐算法

以屏幕窗口内的每个像素为处理单元。确定在每一个像素处，场景中的k个物体哪一个距离观察点最近，从而用它的颜色来显示该像素

```
for (窗口中的每一个像素)  
    {确定距视点最近的物体，  
      以该物体表面的颜色来显示像素;  
    }
```

□ 这类算法是消隐算法的主流！如深度缓冲器算法（Z-buffer算法）、区间扫描线算法等。

□ Idea: 算法的思想是围绕着屏幕的。对屏幕上每个象素进行判断，决定哪个多边形在该象素可见。

消隐分类

(3) 物体空间和图像空间的消隐算法

在物体空间中预先计算面的可见性优先级，再在图像空间中生成消隐图，如画家算法。

隐藏面消隐算法--Hidden Surface Removal

画家算法

Painter's Algorithm

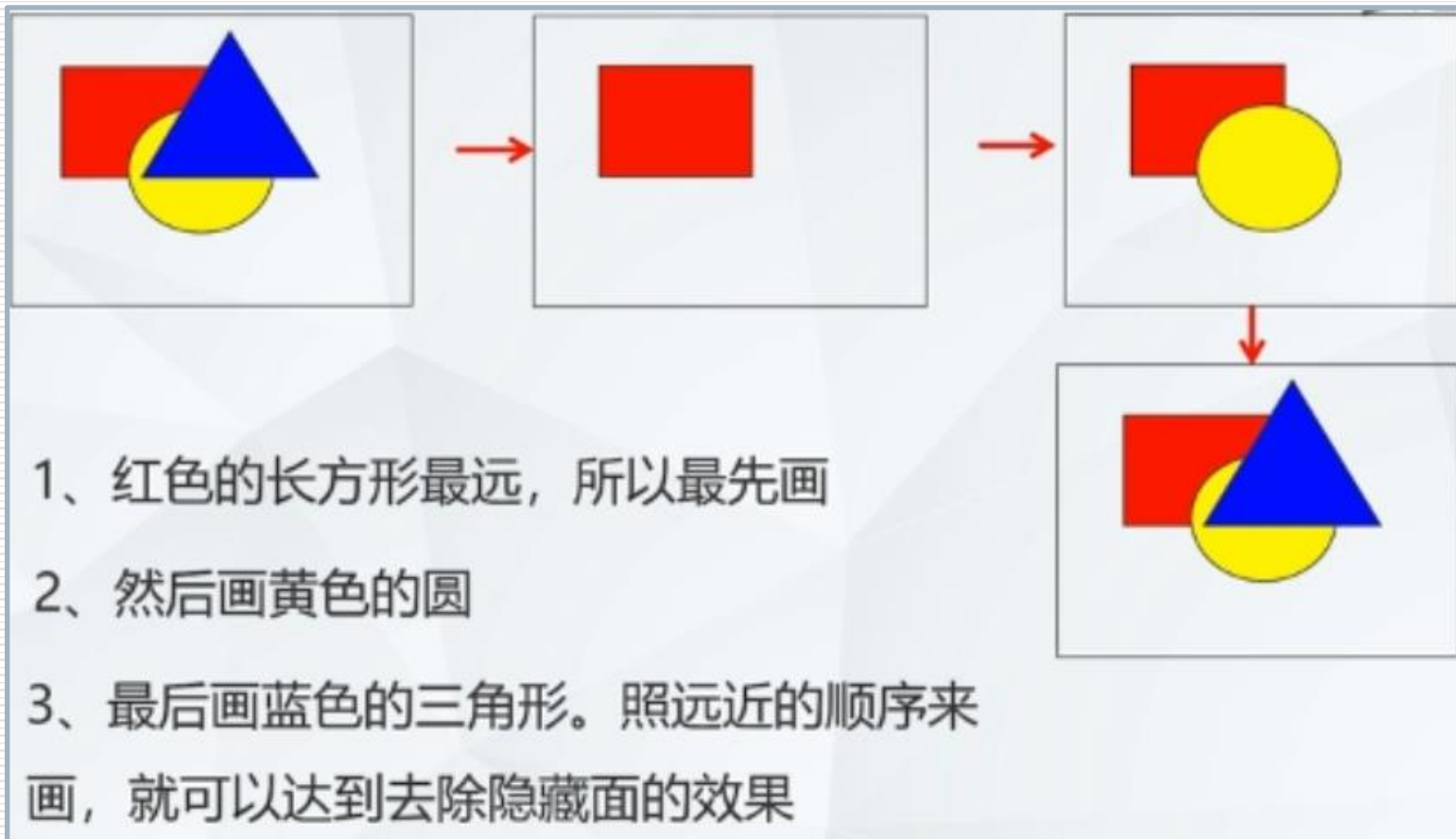
画家算法--Painter's Algorithm

要做到去除隐藏面的最简单方法，就是画家算法

这个方法的原理非常简单。如果一个场景中有许多物体，就是先画远的东西，再画近的东西。这样一来，近的东西自然就会盖住远的东西了

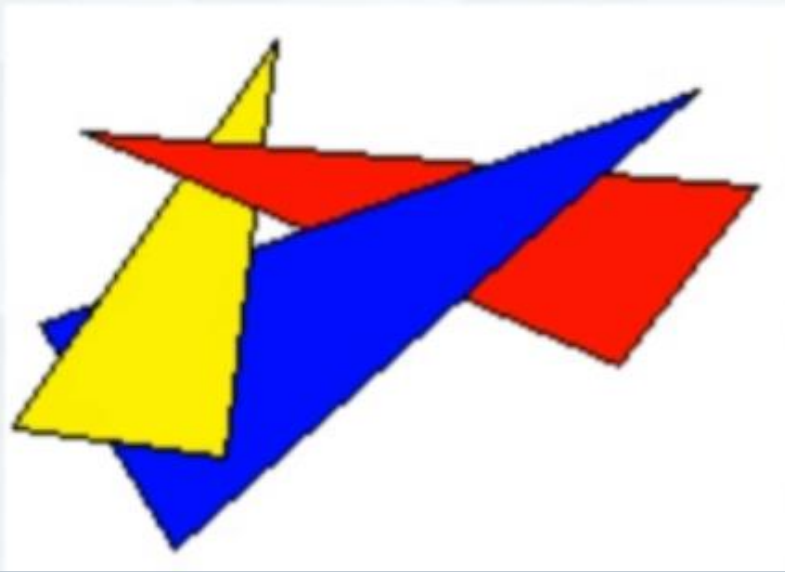


画家算法



画家算法

但实际情况并没有这么理想。在 **三维** 场景中，一个物体可能有些部分远，有些部分近



这个场景里三个三角面互相遮住对方，所以不管用什么顺序去画，都无法得到正确的结果。所以画家算法只能解决简单场景的消隐问题

隐藏面消隐算法--深度缓冲器算法

z-Buffer Algorithm

深度缓冲器算法

--z缓冲器算法，或 **Z-buffer**算法

基于图像空间的主流消隐算法

可解决复杂场景的消隐问题

Z-buffer算法

1973年，犹他大学学生艾德·卡姆尔（Edwin Catmull）独立开发出了能跟踪屏幕上每个像素深度的算法 Z-buffer

Z-buffer让计算机生成复杂图形成为可能。Ed Catmull目前担任迪士尼动画和皮克斯动画工作室的总裁

Z-buffer算法（经典Z-buffer算法）

该算法有帧缓冲器和深度缓冲器。对应两个数组：

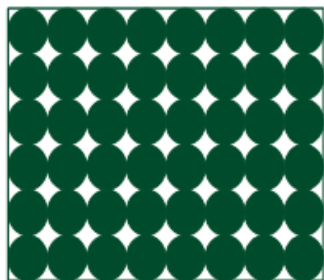
$\text{intensity}(x, y)$ —— 属性数组（帧缓冲器）

存储图像空间每个可见像素的光强或颜色

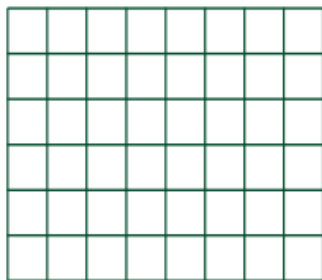
$\text{depth}(x, y)$ —— 深度数组（z-buffer）

存放图像空间每个可见像素的z坐标

屏幕

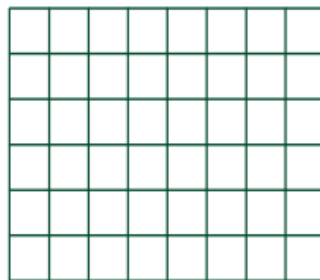


帧缓冲器



每个单元存放对应
像素的颜色值

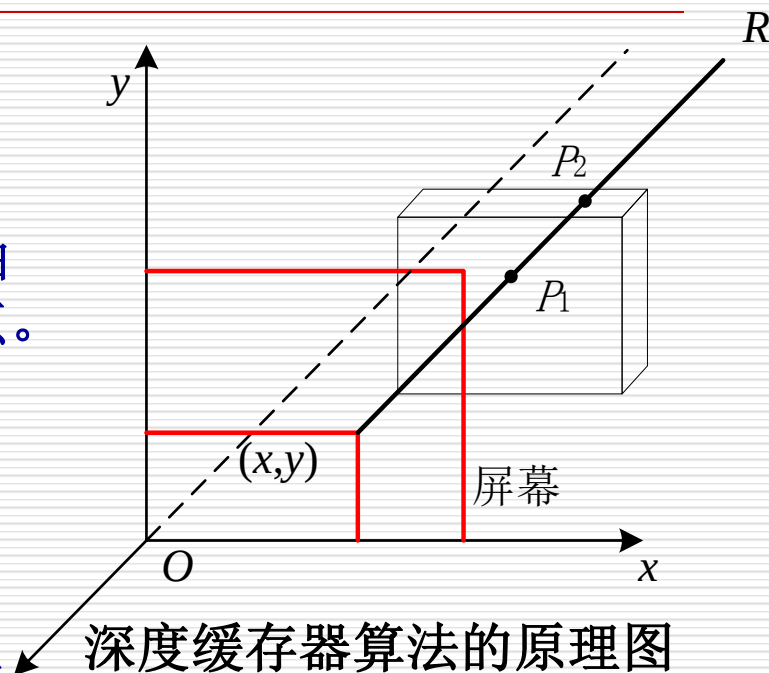
Z缓冲器



每个单元存放对应
像素的深度值

Z-buffer算法 (经典Z-buffer算法)

- 假定 xoy 面为投影面， z 轴为观察方向。视点位于 Z 轴正向无穷远处。
- 过屏幕上任意像素点 (x,y) 作平行于 z 轴的射线 R ，与物体表面相交于 p_1 和 p_2 点。
- p_1 和 p_2 点的 z 值称为该点的深度值。
- **Z-buffer**算法比较 p_1 和 p_2 的 z 坐标值，将最大的 z 坐标值存入 z 缓存中，最大 z 值对应点处的物体表面颜色值存入显示器的帧缓存。
- 显然， p_1 在 p_2 前面，屏幕上 (x,y) 这一点将显示 p_1 点的颜色。



深度缓存器算法的原理图

算法的基本思想：对于屏幕上的每个像素，记录下位于此像素内最靠近观察者的一个景物面的 Z （深度）值和亮度值。

Z-buffer算法步骤(经典Z-buffer算法)

该算法通常应用于表面是平面多边形的物体，因为这些物体适于很快计算出表面深度值，易于实现。曲面可以用多个平面多边形近似表示。

- ◆ 初始化：把Z缓存中各 (x,y) 单元置为 z 的最小值，而帧缓存各 (x,y) 单元置为背景色。
- ◆ 在把物体表面相应的多边形扫描转换成帧缓存中的信息时，对于多边形内的每一采样点 (x,y) 进行处理：
 - 计算采样点 (x,y) 的深度 $z(x,y)$ ；
 - 如 $z(x,y)$ 大于Z缓存中在 (x,y) 处的值，则把 $z(x,y)$ 存入Z缓存中的 (x,y) 处，再把多边形在 $z(x,y)$ 处的颜色值存入帧缓存的 (x,y) 地址中。（此时多边形 (x,y) 处的点比帧缓存中 (x,y) 处现在具有的颜色点更靠近观察者）

Z-buffer算法流程(经典Z-buffer算法)

Z-Buffer算法 ()

{ 帧缓存全置为背景色

深度缓存全置为最小z值

for (每一个多边形)

{ 扫描转换该多边形

for (该多边形所覆盖的每个像素 (x, y))

{ 计算该多边形在该像素的深度值 $Z(x, y)$;

if ($z(x, y)$ 大于z缓存在 (x, y) 的值)

{ 把 $z(x, y)$ 存入z缓存中 (x, y) 处

把多边形在 (x, y) 处的颜色值存入帧缓存的 (x, y) 处

}

}

}

}

Z-buffer算法

□ 如何计算采样点 (x, y) 的深度 $z(x, y)$ 。

■ 假定多边形的平面方程为：

$$Ax + By + Cz + D = 0。$$

$$z(x, y) = \frac{-Ax - By - D}{C}$$

Z-buffer算法 (经典Z-buffer算法)

对于刚刚提到的这个场景，**Z-buffer**算法如何解决的呢？

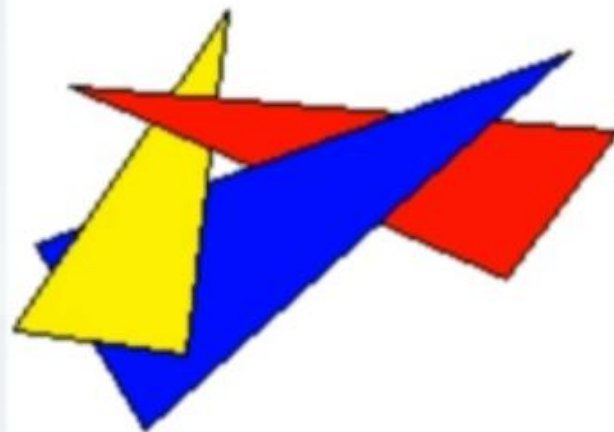
1、先把黄色三角形投影到屏幕上，同时把这个面的每个像素所对应那个点的 z 值存到 z 缓冲区里

2、接着把蓝色的三角形投影到屏幕上，再做扫描转换，转换后有得到一些新的像素，然后再对这些像素进行逐个处理

● 判断一下在原来这个位置是否显示黄颜色，没有就显示蓝颜色

● 如果已经画过黄颜色怎么办？就看谁在前面谁在后面

3、最后再画红色的三角形。重复步骤 2



Z-buffer算法（经典Z-buffer算法）

z-Buffer算法的优点：

- （1）Z-Buffer算法比较简单，也很直观
- （2）在像素级上以近物取代远物。与物体在屏幕上的出现顺序是无关紧要的，有利于硬件实现

z-Buffer算法的缺点：

- （1）占用空间大
- （2）没有利用图形的相关性与连续性，这是z-buffer算法的严重缺陷
- （3）更为严重的是，该算法是在像素级上的消隐算法

改进1:

只用一个深度缓存变量的z-Buffer算法

一般认为，**z-Buffer**算法需要开一个与图象大小相等的缓存数组**ZB**，实际上，可以改进算法，只用一个深度缓存变量**zb**。

```
z-Buffer算法()
{ 帧缓存全置为背景色
  for(屏幕上的每个像素(i, j))  //扫描整个屏幕
  { 深度缓存变量zb置最小值MinValue
    for(多面体上的每个多边形Pk)
    {
      if(像素点(i, j)在pk的投影多边形之内)
      {
        计算Pk在(i, j)处的深度值depth;
        if(depth大于zb)
        { zb = depth;
          indexp = k; (记录多边形的序号)
        }
      }
    }
    If(zb != MinValue) 计算多边形Pindexp在交点 (I, j) 处的光照
                        颜色并显示
  }
}
```

点与多边形的包含性检测

- 关键问题：判断像素点 (i,j) 是否在 pk 的投影多边形之内，不是一件容易的事。节省了空间但牺牲了时间。计算机的很多问题就是在时间和空间上找平衡。

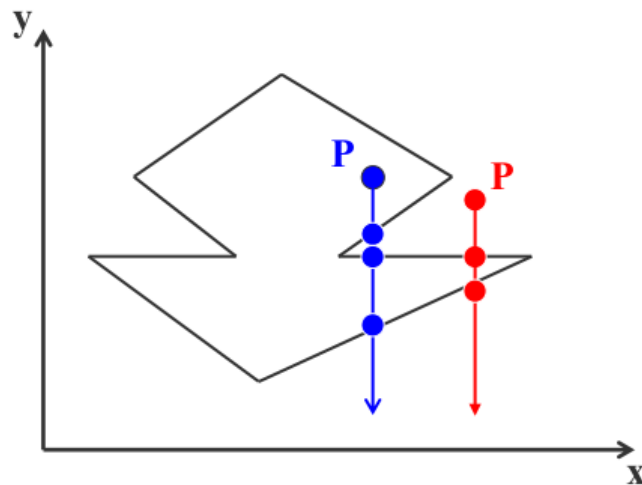
点与多边形的包含性检测：

(1) 射线法

由被测点 P 处向 $y = -\infty$ 方向作射线

交点个数是奇数，则被测点在多边形内部

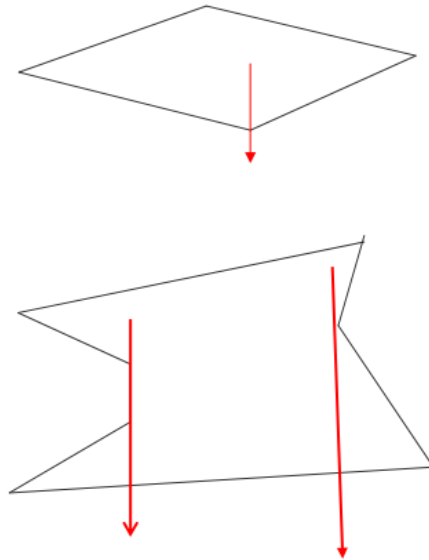
交点个数是偶数表示在多边形外部



射线法

若射线正好经过多边形的顶点，则采用“左开右闭”的原则来实现

即：当射线与某条边的顶点相交时，若边在射线的左侧，交点有效，计数；若边在射线的右侧，交点无效，不计数

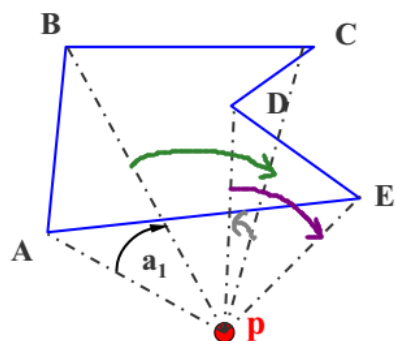


用射线法来判断一个点是否在多边形内的弊端：

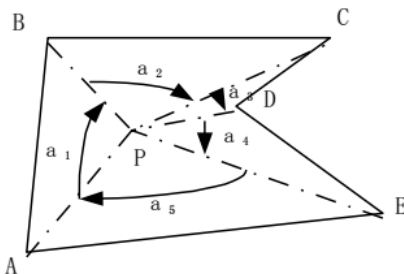
- (1) 计算量大
- (2) 不稳定

弧长法

(2) 弧长法



以p点为圆心，作单位圆，把边投影到单位圆上，对应一段段弧长，规定逆时针为正，顺时针为负，计算弧长代数和



代数和为**0**，点在多边形外部
代数和为 **2π** ，点在多边形内部
代数和为 **π** ，点在多边形边上

这个算法为什么是稳定的？假如算出来后代数和不是**0**，而是**0.2**或**0.1**，那么基本上可以断定这个点在外部，可以认为是有计算误差引起的，实际上是**0**。

改进2: 扫描线Z-buffer算法(了解)

前面介绍了经典的z-buffer算法，思想是开一个和帧缓存一样大小的存储空间，利用空间上的牺牲换取算法上的简洁

还介绍了只开一个缓存变量的z-buffer算法，是把问题转化成判别点在多边形内，通过把空间多边形投影到屏幕上，判别该像素是否在多边形内

□ **z-buffer**算法是非常经典和重要的，在图形加速卡和固件里都有。

只用一个深度缓存变量**zb**的改进算法虽然减少了空间，但仍然没考虑图形相关性和连贯性。

改进2:

扫描线Z-buffer算法(了解)

- **z-buffer**算法是非常经典和重要的，在图形加速卡和固件里都有。只用一个深度缓存变量**zb**的改进算法虽然减少了空间，但仍然没考虑图形相关性和连贯性。
- 可以利用连贯性加速深度的计算。
- 为了充分利用物体表面沿**相邻扫描线和相邻像素的连贯性**，可利用前面介绍的边表和有效边表提高算法效率。一般将构造了有效边表的**Z-buffer**算法称为**扫描线Z-buffer**算法。
- **扫描线Z-buffer**算法它把处理单元从象素点扩展到扫描线（即同一水平象素点的集合）